

COMPLETE INTERVIEW PREP EDITION

System Design Mastery

The complete, battle-tested study plan: a 30-day master plan, core concept modules with notes, a practice-problem bank, guided build projects, large-scale system designs, and a full interview-prep playbook.

One-time purchase · lifetime updates · 30-day money-back guarantee

Contents

01. Getting Started
02. The 30-Day Master Plan
03. Practice Problem Bank
04. Build Projects
05. Resources & References

06. Core Concepts
07. Guided Build Projects
08. Large-Scale System Designs
09. Advanced Deep Dives
10. Interview Prep Playbook

01. Getting Started

System Design Mastery

System Design Mastery

An open-source, complete system design study plan — from fundamentals to large-scale builds, with a concrete **interview-prep master plan**.

Start here

- **Master Plan** — [the full strategy, day-by-day](#) → read this first
- [Practice Problems](#) — 16 problems by tier
- [Build Projects](#) — 10 hands-on projects

The 6 phases

1. **Foundations** — scaling, networking, databases, caching
2. **Core Concepts** — load balancing, message queues, consistent hashing, CAP
3. **Classic Designs** — URL shortener, rate limiter, chat, notifications
4. **Large-Scale Systems** — news feed, Instagram, YouTube
5. **Advanced Systems** — Google Docs, Maps, Web Crawler, etc.
6. **Interview** — framework, practice platforms, FAQ bank

Browse the sidebar for every topic. Every page is freely editable via GitHub.

Support this project

This entire course is **free and open source** — notes, practice, and projects are all in the repo.

If it helped you land a job, pass an interview, or level up, you can buy the **premium edition** on Gumroad as a one-time donation (lifetime updates, same content, extra convenience):

Buy System Design Mastery — Complete Interview Prep — \$49 one-time

Every purchase directly supports keeping this project free and improving it.

02. The 30-Day Master Plan

 **System Design Master Study Plan — START HERE**

System Design Master Study Plan — START HERE

If you only read one file in this repo, read this one. Everything else in this repo is study *material*. This file is the *strategy*: what to study, in what order, how much per day, where to practice, and how to get interview-ready.

1. The One-Number Goal

You are interview-ready when you can, under time pressure, produce a complete high-level design for an unfamiliar system in 45 minutes — requirements → estimates → diagram → schema → scaling — while explaining every trade-off.

That's it. 90% of candidates fail because they knew concepts but couldn't *produce a design on the spot*. This plan trains production.

2. How Much Time Per Day

Situation	Daily time	Ready in
Full-time job hunting (8h/day)	5-6h focused	~4 weeks
Working + studying (typical)	2-3h + weekends	~8-10 weeks
Casual / no deadline	1h/day	~12-16 weeks

Non-negotiable daily minimum: 1 pomodoro-pair (2 × 45 min) on weekdays + 4h on weekends. Skipping days is what kills candidates.

3. The 3-Track System

Never study only concepts. Run all three tracks in parallel, every week:

TRACK A: LEARN	(concepts → 40% of time)
Read repo modules in order. Write your own notes.	
TRACK B: PRACTICE	(design out loud → 45% of time)
Pick a problem, design it to a timer, ALWAYS out loud.	
TRACK C: MOCK	(realistic reps → 15% of time)
Live mock interviews with feedback. Starts Week 2, every week.	

4. The 4-Phase Timeline (maps to repo structure)

PHASE 1 — Foundations (Days 1–8)

Learn: every module in 01-Fundamentals → 04-Caching, in order.

Day	Study (Track A)	Practice (Track B)	Mock (Track C)
1	Scaling, latency/throughput numbers (01-Fundamentals/)	Sketch a key-value store	—
2	DNS, TCP/UDP, HTTP/HTTPS, WebSocket, CDN (02-Networking/)	Design a static website → CDN flow	—
3	SQL vs NoSQL, ACID/BASE, indexing (03-Databases/)	Pick DB for a URL shortener, justify	—
4	Caching: cache-aside, TTL, invalidation (04-Caching/)	Draw cache layer for a read-heavy feed	—
5	REVIEW + WRITE DOWN all numbers from memory (estimation sheet below)	Mini-design: book catalog	—
6	FIRST FULL DESIGN: Design URL Shortener (09-Projects/URL-Shortener/) — READ it, then re-produce from memory out loud	Same problem, timer 45 min	—
7	Fix gaps from Day 6. Read 01-Fundamentals/ once more	Re-do design with improvements	—
8	Assessment quiz (write on paper, no looking): terms, trade-offs, numbers	Re-do design cold	[BOOK FIRST MOCK for next week]

Success checkpoint: You can design URL Shortener out loud from scratch, with numbers, in 45 min. **Fail?** Repeat Days 6-8 before moving on. Do NOT rush foundations.

PHASE 2 — Core Concepts (Days 9–16)

Learn: 05-LoadBalancing → 08-CAPTheorem .

Day	Study	Practice
9	Load balancing: L4/L7, algorithms, health checks	Add LB layer to URL Shortener
10	Message queues: Kafka topic model, patterns	Design async order processing
11	Consistent hashing + partitioning	Draw ring for 5-node cache, add node
12	CAP + PACELC, consistency models, quorum	Classify 5 real DBs (Cassandra, Mongo, Spanner, Dynamo, Postgres)
13	FULL DESIGN #2: Rate Limiter (09-Projects/Rate-Limiter/)	Timer design out loud
14	FULL DESIGN #3: Chat System (09-Projects/Chat-System/)	Read, then reproduce cold
15	Write your own cheat-sheet combining Phases 1-2	Re-design worst one from memory
16	Assessment + MOCK #1 (live, with feedback)	Fix mock gaps same day

Mock #1 inputs: read 12-InterviewPrep/README.md (the 4-step framework) BEFORE the mock.

PHASE 3 — Large-Scale Builds (Days 17–28)

Learn & reproduce: every project in 10-LargeScale/ .

Day	Problem (all → read, then reproduce cold out loud)
17	Twitter/News Feed (10-LargeScale/Twitter-Feed/)
18	Twitter/News Feed — second pass with numbers
19	MOCK #2 on News Feed
20	Instagram/Photo Sharing (10-LargeScale/Instagram/)
21	Instagram — second pass
22	YouTube/Video Streaming (10-LargeScale/YouTube/)
23	MOCK #3 on Instagram or YouTube
24	Rest day or re-do weakest
25	Notification System (09-Projects/Notification-System/)
26	Google Docs / real-time collab (11-Advanced/Google-Docs/)
27	Pick wildcard: Google Maps / Web Crawler (11-Advanced/)
28	MOCK #4 (ask interviewer for a problem you haven't studied)

PHASE 4 — Polish & Mock Marathon (Days 29+)

- **2 full designs per day**, alternating studied + unseen problems.
 - **2-3 mocks per week** (Pramp / interviewing.io — cheap or free). Always practice, debrief 15 min, fix one gap.
 - One day/week: rehearse **estimation** + **schema-only speed drills** (5 min each).
 - Last week: **only timed full designs** + review your cheat-sheet + interview-day playbook.
-

5. The Numbers You Must Memorize (estimation cheat sheet)

Print these. Recite daily until automatic:

```
Scales:  K=103  M=106  B=109  T=1012
```

Requests:

```
1M req/day ≈ 12 QPS           100M req/day ≈ 1,200 QPS
30K req/sec ≈ 2.6B/day       peak ≈ 2× average
```

Latency (read: millis):

```
L1 cache ~0.5ns | RAM ~100ns | SSD ~0.1ms | HDD ~10ms
Same DC network ~0.5ms | cross-zone ~1-2ms | cross-region ~100ms
TLS handshake ~2xRTT | DNS ~1ms-100ms
```

Storage bytes:

```
char=2B | int=4B | long=8B | tiny URL record=500B
tweet text≈280B | 1080p 1hr video≈1.5-4GB | photo=500KB
```

Throughput:

```
MySQL/PG: ~1K-10K writes/s, reads w/ replicas scale out
Redis: ~100K ops/s single node
Kafka: ~100K-1M msgs/s cluster
1Gbps link ≈ 125MB/s
```

Retention:

```
5 years × 180B URLs × 500B ≈ 90TB
500hrs video/min × 60 × 24 ≈ 720K hrs/day new video
```

6. Where to Practice Frequently-Asked Questions

Where	What	Cost	Use it for
Pramp	Peer mock interviews (scheduled)	Free	Weekly Track C mocks — best free option
interviewing.io	Anonymous mocks, real company interviews recorded	Freemium	Realistic pressure + watch recorded examples
LeetCode System Design (discuss tab, tagged questions)	Text-based design Q&A	Free/paid	Reading model answers + seeing common questions
Alex Xu's Grokking	16 guided problems	Paid	Golden sample answers AFTER you attempt yourself
ByteByteGo	Diagrams/animated course + mock service	Paid	Visuals + mock interviews with feedback
this repo / PRACTICE.md	16 prioritized problems w/ difficulty	Free	Your daily Track B rotation
YouTube: Gaurav Sen , ByteByteGo , codeKarle	Walkthroughs	Free	Watch ONE per problem AFTER your attempt (not before)

Golden rule: Never read/watch a solution before attempting the design yourself for 30+ min. Attempt → struggle → compare → note the delta. That delta is where you improve.

7. Frequently Asked Questions — Priority Bank (do these in order)

#	Problem	Priority	Difficulty	Study in repo
1	URL Shortener	MUST	Easy	09-Projects/URL-Shortener/
2	Rate Limiter	MUST	Easy	09-Projects/Rate-Limiter/
3	News Feed (Twitter)	MUST	Medium	10-LargeScale/Twitter-Feed/
4	Chat / WhatsApp / Slack	MUST	Medium	09-Projects/Chat-System/
5	Instagram / Photo Sharing	MUST	Medium	10-LargeScale/Instagram/
6	Notification System	HIGH	Medium	09-Projects/Notification-System/
7	YouTube / Video Streaming	HIGH	Hard	10-LargeScale/YouTube/
8	Google Docs / Collab	HIGH	Hard	11-Advanced/Google-Docs/
9	Uber / Lyft (matching)	MED	Hard	analogy: 11-Advanced/Google-Maps/
10	Google Maps / geospatial	MED	Hard	11-Advanced/Google-Maps/
11	E-commerce / Amazon	HIGH	Hard	design your own (see PRACTICE.md)
12	Web Crawler	MED	Hard	11-Advanced/Web-Crawler/

#	Problem	Priority	Difficulty	Study in repo
13	Google Search	MED	Hard	PRACTICE.md #15
14	Distributed Lock	LOW	Hard	PRACTICE.md #16
15	Typeahead / Autocomplete	MED	Medium	PRACTICE.md #14
16	Leaderboard	LOW	Medium	PRACTICE.md

Why is the "MUST" set #1-5? They're asked at 70%+ of companies, and they teach every pattern you need (read-heavy feed, write-heavy streaming, real-time, media pipeline). Master #1-5 cold and you can *learn* any of the others live in the room.

8. The Mock Interview Cadence (non-negotiable)

```

Week 1 : -
Week 2 : MOCK #1 (URL Shortener or Rate Limiter)
Week 3 : MOCK #2 (News Feed)
Week 4 : MOCK #3 (unseen problem – pick it yourself)
Week 5+ : 2-3 mocks per week, mix seen/unseen
Week 6+ : Same. YES, even before you "feel ready."
```

Why this early? Mocks surface *process* problems (rambling, no numbers, skipping requirements) that reading never reveals. You can't fix what you never see.

9. Interview-Day Playbook

24–48 hours before

- Rehearse **2 full designs** out loud; one studied, one new.
- Re-read 12-InterviewPrep/README.md framework.
- Print numbers sheet + your cheat-sheet.

Night before

- **No new topics.** Sleep >7h. (A well-rested D-average beats a sleep-deprived A+.)
- Lay out clothes + water. Confirm time/link/recorder.

Morning of

- Light review only (cheat-sheet, 1 warm-up design).
- Coffee/water, brain food, brisk 10-min walk if tense.

During (the 45-min flow)

00:00-05 Requirements: ask 4-6 clarifying Qs. WRITE them down. Agree scope.
05:10 Estimation: 3-5 measures (QPS, storage, latency). State assumptions.
10:25 High-level: diagram main boxes + arrows + data flow. Keep simple.
25:40 Deep dive: bottleneck, cache, schema, failure modes. Explain trade-offs.
40:45 Summary: recap design + 2 improvements you'd make if you had time.

Red flags that kill candidate (avoid)

Red flag	Fix
Design without clarifying scope	Always ask: users? reads vs writes? consistency?
No numbers anywhere	Estimations = proof of seniority
One-word answers, no trade-offs	Say "X gives A but costs B; alternative is C"
Silent 5+ min	Talk constantly, even to think out loud
Building the whole thing at once	4-step framework, one box at a time
Ignoring failure modes	Every component: "if this dies, what happens?"
Over-engineering first pass	Simple + scaling plan, not a monolith of every idea

10. Progress Tracker (tick as you go)

FOUNDATIONS

- [] Day 1-2 Scaling + networking
- [] Day 3-4 DB + caching
- [] Day 5 Numbers memorized
- [] Day 6-8 URL Shortener cold, 45 min

CORE CONCEPTS

- [] Day 9-10 LB + queues
- [] Day 11-12 hashing + CAP/PACELC
- [] Day 13 Rate Limiter cold
- [] Day 14 Chat System cold
- [] Day 15 cheat-sheet written
- [] Day 16 MOCK #1 done + gaps fixed

LARGE-SCALE

- [] Day 17-18 News Feed cold
- [] Day 19 MOCK #2 (News Feed)
- [] Day 20-21 Instagram cold
- [] Day 22-23 YouTube + MOCK #3
- [] Day 25 Notification System
- [] Day 26-27 Docs / Maps / Crawler
- [] Day 28 MOCK #4 (unseen problem)

POLISH

- [] 2 designs/day routine running
- [] 2-3 mocks/week sustained
- [] Estimation + schema speed drills
- [] Interview-day playbook rehearsed
- [] Got the offer 🎉

11. One-Week-Before-Interview Checklist

- ☐ You can produce **5 of the 7 MUST/HIGH problems** cold in 45 min
- ☐ Numbers sheet automatic (recite without looking)
- ☐ At least **4 live mocks** completed (2+ with external feedback)
- ☐ Cheat-sheet fits on one page and you used it in mocks
- ☐ You know the 4-step framework by heart and use it every time
- ☐ You practice out loud daily (voice memo your 2 daily designs)

12. Final Truth

Reading this repo makes you *informed*. Designing out loud to a timer, getting mocked, and fixing gaps makes you *hired*. **Track B and C are the plan**. Everything else is fuel.

Good luck. Start with Day 1. Tomorrow.

13. Support This Plan

This course is free and open source. If this plan helped you land an offer, the best way to say thanks (and keep it free) is to **buy the premium edition on Gumroad** 💎 \ one-time, lifetime updates, same content.

Every sale funds more free content.

03. Practice Problem Bank

Practice Problems (Interview Prep)

Practice Problems (Interview Prep)

Beginner Practice (Weeks 1-8)

Problem 1: Design a URL Shortener

Key Concepts: Hashing, storage, caching, redirects, Base62

Requirements:

- Shorten long URLs
- Handle 100M URLs/day
- Click analytics
- Custom aliases

Iterate On:

- ☐ Requirements clarification
- ☐ Traffic estimation
- ☐ High-level diagram
- ☐ Database schema
- ☐ Cache strategy
- ☐ Scaling plan

Problem 2: Design a Rate Limiter

Key Concepts: Algorithms (token bucket, sliding window), Redis, distributed systems

Requirements:

- Limit requests per IP/user
- Handle distributed servers
- Different limits per API
- Graceful 429 responses

Problem 3: Design a Load Balancer

Key Concepts: L4 vs L7, health checks, session affinity

Requirements:

- Distribute traffic
- Handle server failures

- Support HTTP + HTTPS
- Stickiness for sessions

Problem 4: Design a Distributed Cache

Key Concepts: Consistent hashing, LRU/LFU, TTL, replication

Requirements:

- High throughput reads
- Cache invalidation
- Node add/remove
- Handle cache stampede

Intermediate Practice (Weeks 9-16)

Problem 5: Design a Chat Messaging App (WhatsApp/Slack)

Key Concepts: WebSockets, presence, message ordering, offline support

Requirements:

- One-on-one and group chat
- Message delivery (real-time)
- Read receipts
- Media sharing
- Offline message queue

Extend With:

- Group chat (50 vs 1000 members)
- Video/audio calls
- Message encryption

Problem 6: Design a News Feed (Twitter/Facebook)

Key Concepts: Fan-out, timelines, social graph, caching

Requirements:

- Post content (text, images, video)
- Follow/unfollow
- Personalized timeline
- Like/comment/share
- Trending topics

Extend With:

- Ad insertion
- User engagement algorithms

- Search within feed

Problem 7: Design a Video Platform (YouTube)

Key Concepts: Media pipeline, transcoding, CDN, adaptive bitrate

Requirements:

- Video upload
- Video streaming
- Recommended videos
- Search
- Comments/likes

Extend With:

- Live streaming
- Video editing in browser
- Monetization

Problem 8: Design Instagram (Photo Sharing)

Key Concepts: Image storage, CDN, feed, caching

Requirements:

- Upload photos
- Filters
- Feed with images
- Follow system
- Likes/comments

Extend With:

- Stories (24hr)
- Reels (short video)
- Direct messages

Advanced Practice (Weeks 17-24)

Problem 9: Design Google Docs

Key Concepts: OT, CRDTs, WebSockets, version control

Requirements:

- Multiple concurrent editors
- Real-time
- Offline support
- Version history

- Comments

Problem 10: Design Uber/Lyft

Key Concepts: Geospatial, matching, real-time, load balancing

Requirements:

- Driver/rider locations
- Ride matching
- Fare calculation
- Navigation routing
- Surge pricing

Trade-offs To Discuss:

- Real-time location updates (frequency?)
- Driver allocation (greedy vs global optimization?)
- Consistency of ride status

Problem 11: Design Google Maps

Key Concepts: Geospatial indexing, tiling, routing algorithms

Requirements:

- Map rendering (tiles)
- Location search
- Directions + navigation
- Traffic + live data

Problem 12: Design E-commerce (Amazon)

Key Concepts: Transactions, inventory, payments, idempotency

Requirements:

- Product catalog
- Shopping cart
- Order placement
- Payments
- Order tracking
- Recommendations

Key Concerns: Distributed transactions, inventory consistency, payment idempotency

Problem 13: Design a Web Crawler (Google)

Key Concepts: URL frontier, politeness, deduplication, distributed processing

Requirements:

- Crawl entire web
- Respect robots.txt
- Handle dynamic content
- Deduplicate
- Prioritize URLs

Problem 14: Design a Notification System

Key Concepts: Push/pull, multi-channel, queues

Requirements:

- Multiple channels: push, SMS, email
- User preferences
- Delivery retries
- Zero message loss

Problem 15: Design a Search Engine (Google)

Key Concepts: Indexing, query processing, ranking

Requirements:

- Crawl web
- Index documents
- Search queries
- Ranking
- Autocomplete

Problem 16: Design a Distributed Lock

Key Concepts: Redlock, Zookeeper, consensus

Requirements:

- Acquire/release locks
- Lock expiration
- Fault tolerance
- Deadlock prevention

Cheat Sheet: Quick Design Templates

Read-Heavy System (Twitter, Instagram)

Client → CDN → LB → App Servers → Cache → DB (Read Replicas)

↑

Write Master → Async Jobs → Analytics

Write-Heavy System (Logging, Analytics)

```
Clients → Ingestion API → Kafka → Stream Processor → Storage
                                     ↓
                               Consumers (multiple)
```

Real-time System (Chat, Notifications)

```
Client (WebSocket) ↔ WS Server ↔ Message Brokers
                                     ↓
                               Persistence (offline)
                               Pub/Sub (presence)
```

Media System (YouTube, Netflix)

```
Upload → Storage → Transcoding Pipeline → CDN
                                     ↓
                               Adaptive Streaming
```

Practice Routine

Weekly Practice (1 hour)

- ☐ 15 min: Requirements + estimation
- ☐ 25 min: Full high-level design (talking out loud)
- ☐ 10 min: Identify 2 bottlenecks, propose fixes
- ☐ 10 min: Self-review against checklist

Bi-Weekly Mock (2 hours)

- ☐ Full 45-min mock interview
- ☐ Feedback review
- ☐ Update notes with mistakes
- ☐ Retry weak problem after 1 week

Resources

- [Pramp - Free mock interviews](#)
- [LeetCode System Design](#)
- [Grokking System Design](#)
- [Gaurav Sen's System Design Series](#)

04. Build Projects

Implementation Projects

Implementation Projects

Hands-on projects to implement core system design concepts. Start simple, then scale up.

Beginner Projects

Project 1: CLI URL Shortener

Goal: Understand hashing, storage, and basic web APIs

Requirements:

- CLI tool to shorten URLs
- Storage in SQLite or JSON file
- Basic redirect server (Flask/Express)

Skills Gained: HTTP, database, basic algorithms

Checklist:

- ☐ Implement Base62 encoding
- ☐ Store and retrieve URLs
- ☐ Build HTTP redirect endpoint
- ☐ Add click tracking

Project 2: Rate Limiter Middleware

Goal: Implement rate limiting algorithms from scratch

Requirements:

- Token bucket implementation
- Sliding window implementation
- Redis-backed (distributed)
- API middleware integration

Skills Gained: Caching, distributed systems, concurrency

Checklist:

- ☐ Implement token bucket
- ☐ Implement sliding window
- ☐ Add Redis storage
- ☐ Response headers (X-RateLimit-*)

- ☐ Handle distributed edge cases

Project 3: Distributed Key-Value Store

Goal: Build a simple distributed key-value store

Requirements:

- GET/SET/DEL operations
- Consistent hashing
- Replication
- Fault tolerance

Skills Gained: Distributed systems, replication, consensus

Checklist:

- ☐ Single-node implementation
- ☐ Add consistent hashing
- ☐ Add replication
- ☐ Leader election
- ☐ Failure recovery

Intermediate Projects

Project 4: News Feed API

Goal: Implement the core of a Twitter-like feed

Requirements:

- Post messages
- Follow users
- Timeline generation (fan-out on write)
- Feed pagination
- Cache with Redis

Stack: Node/Python + PostgreSQL/Redis + Kafka

Checklist:

- ☐ User and follow tables
- ☐ Post endpoint
- ☐ Timeline merge logic
- ☐ Redis timeline cache
- ☐ Fanout on write
- ☐ Feed pagination

Project 5: Real-time Chat Application

Goal: Build WebSocket-based chat with presence

Requirements:

- WebSocket connections
- Message history
- Online/offline status
- Multiple rooms
- Delivery receipts

Stack: Socket.io/WebSocket + Redis Pub/Sub

Checklist:

- ☐ WebSocket server
- ☐ Multiple rooms
- ☐ Message persistence
- ☐ Presence tracking
- ☐ Read receipts
- ☐ Scale across multiple servers (Redis Pub/Sub)

Project 6: E-commerce Cart + Checkout

Goal: Design transactional flows with distributed concerns

Requirements:

- Shopping cart
- Inventory management
- Order processing
- Payment integration (idempotency)
- Order status tracking

Checklist:

- ☐ Cart service
- ☐ Inventory with optimistic locking
- ☐ Order state machine
- ☐ Payment idempotency
- ☐ Message queue for order processing
- ☐ Failure handling

Advanced Projects

Project 7: Streaming Data Pipeline

Goal: Build a real-time analytics pipeline

Requirements:

- Ingest events via HTTP
- Emit to Kafka
- Stream processing (aggregation)
- Dashboard updates
- Backpressure handling

Stack: Producer + Kafka + Consumer group + Dashboard

Checklist:

- ☐ Event ingestion API
- ☐ Kafka producer/consumer
- ☐ Windowed aggregation
- ☐ Dashboard with real-time updates
- ☐ Fault tolerance for consumers
- ☐ Exactly-once semantics (or at-least-once with dedup)

Project 8: Distributed Cache System

Goal: Build Redis-like cache with partitioning

Requirements:

- Consistent hashing
- Replication (read + write)
- LRU/LFU eviction
- TTL support
- Client library

Checklist:

- ☐ Hash ring with virtual nodes
- ☐ Replication protocol
- ☐ Eviction policies
- ☐ TTL implementation
- ☐ Node add/remove without data loss
- ☐ Client SDK

Project 9: Search Engine (Mini Google)

Goal: Implement search with indexing and ranking

Requirements:

- Crawler
- Indexer (inverted index)
- Query search → results
- Basic ranking

Checklist:

- ☐ Web crawler (politeness)
- ☐ Inverted index
- ☐ TF-IDF ranking
- ☐ Query processing
- ☐ Pagination
- ☐ Optimization (caching, sharding)

Capstone Project

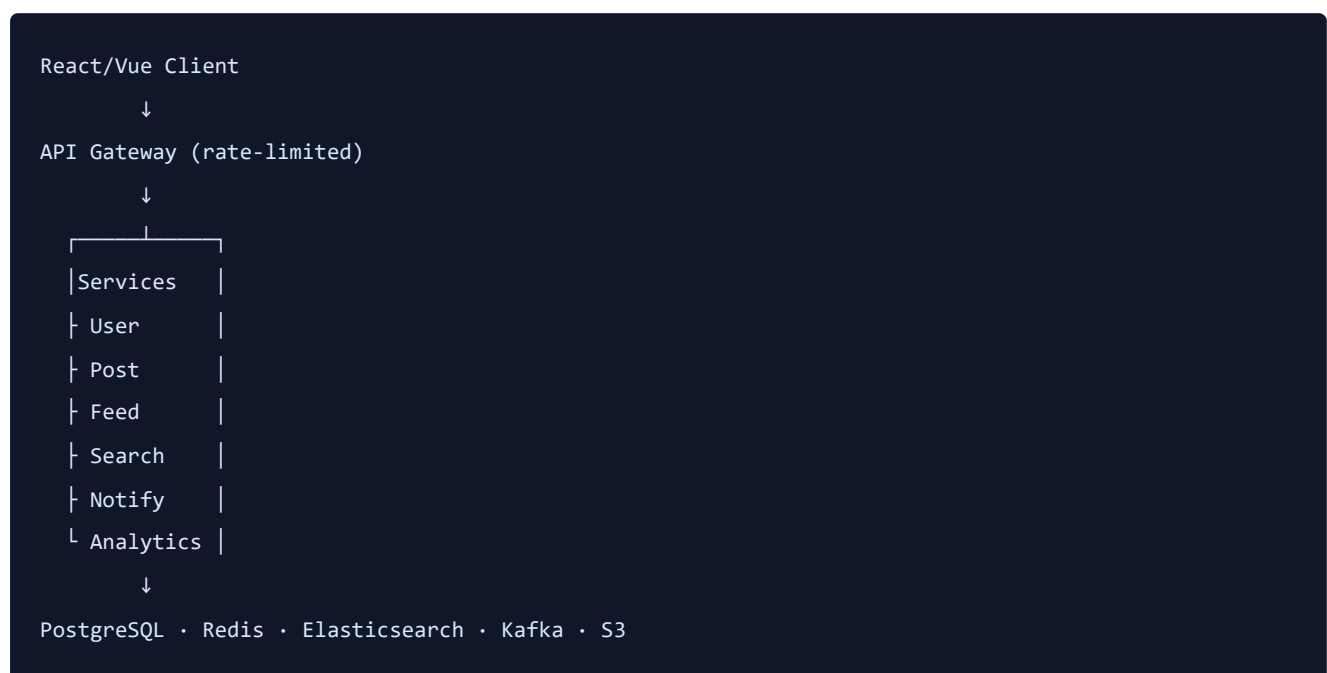
Project 10: Full-Stack Social Platform

Goal: Everything learned, in one platform

Requirements: A mini-Twitter with:

- User auth
- Post + media upload
- Timeline (hybrid fanout)
- Search
- Notifications
- Analytics dashboard
- Rate limiting
- Caching everywhere

Architecture:



Checklist:

- ☐ Setup monorepo
- ☐ Docker containers
- ☐ CI/CD pipeline
- ☐ Authentication
- ☐ All services
- ☐ Testing
- ☐ Deployment
- ☐ Monitoring (Prometheus/Grafana)
- ☐ Load testing (k6)

Deployment Recommendations

Component	Tool
Containers	Docker
Orchestration	Docker Compose → Kubernetes
CI/CD	GitHub Actions
Monitoring	Prometheus + Grafana
Logging	Loki / ELK
Tracing	Jaeger / Zipkin
Load testing	k6 / JMeter

05. Resources & References

Curated System Design Resources

Curated System Design Resources

Hand-picked best resources from across the web for learning system design.

Books (Must Read)

Book	Author	Focus
Designing Data-Intensive Applications	Martin Kleppmann	★ THE foundational book - distributed systems, databases, consistency
System Design Interview - An Insider's Guide (Vol 1)	Alex Xu	★ Best interview prep with 15 real problems
System Design Interview (Vol 2)	Alex Xu	Advanced problems like Kafka, Google Maps
Designing Distributed Systems	Brendan Burns	Kubernetes author's patterns
Understanding Distributed Systems	Roberto Vitillo	Free online book, practical
Site Reliability Engineering	Google	Production systems perspective

Online Courses

Course	Platform	Cost	Why
Grokking the System Design Interview	Educative	Paid	★ Most popular for interviews
System Design Interview Course	ByteByteGo	Paid	★ By Alex Xu, best diagrams
MIT 6.824: Distributed Systems	MIT	Free	★ The gold standard CS course
Intro to Distributed Systems	Udacity	Free	Georgia Tech
AWS Architecture Center	AWS	Free	Real-world reference architectures
Grokking Advanced System Design	Educative	Paid	Distribute-2 style microservices

Free Resources

GitHub Repos

- [System Design Primer](#) - ★ 220K+ stars, THE free resource
- [System Design 101](#) - Visual explanations

- [Awesome System Design](#) - Curated list
- [System Design Interview Repository](#) - Interview questions
- [Design Patterns for Distributed Systems](#)
- [System Design Cheatsheet](#) - Single-page reference

YouTube Channels

- [ByteByteGo](#) - ★ Best for quick visual explanations
- [Gaurav Sen](#) - ★ Great deep dives into algorithms
- [codeKarle](#) - System design interview prep
- [System Design Interview](#) - Detailed walkthroughs
- [Jordan has no life](#) - Advanced system design
- [InfoQ](#) - Real-world talks from tech companies

Blogs & Articles

- [The System Design Primer Blog](#)
- [Martin Fowler's Website](#) - Enterprise architecture patterns
- [All Things Distributed](#) - Werner Vogels (AWS CTO)
- [High Scalability](#) - ★ Real-world architectures (archived articles still valuable)
- [Confluent Blog](#) - Kafka and streaming
- [Netflix Tech Blog](#) - Production system design
- [Uber Engineering Blog](#) - Real architecture decisions

Practice Resources

Mock Interview Platforms

- [Pramp](#) - Free peer mock interviews
- [Interviewing.io](#) - Anonymous mock interviews
- [ByteByteGo Mock Interview](#) - Practiced by Alex Xu
- [System Design Mock Interviews](#) - Payment based

Practice Problems (Classic)

1. **URL Shortener** (like bit.ly, TinyURL)
2. **Rate Limiter**
3. **Notification System**
4. **Chat Messaging System** (WhatsApp, Slack)
5. **News Feed** (Twitter, Facebook)
6. **Photo Sharing** (Instagram)
7. **Video Streaming** (YouTube, Netflix)
8. **Real-time Collaboration** (Google Docs)

9. **Location Service** (Google Maps)
10. **E-commerce** (Amazon)
11. **Search Engine** (Google)
12. **Web Crawler**
13. **Book Sharing** (in-person based)
14. **Typeahead/Autocomplete**
15. **Leaderboard** (gaming)
16. **Distributed Lock**
17. **Logging System**
18. **Distributed Cache**
19. **Payment System** (Stripe)
20. **Hotel Booking** (Airbnb)

Problem Solving Sites

- [LeetCode System Design](#)
- [System Design Questions - LinkedIn](#)
- [GeeksforGeeks System Design](#)

Company Engineering Blogs (Real Architectures)

Social Media

- [Twitter Engineering](#)
- [Meta \(Facebook\) Engineering](#)
- [YouTube Engineering Blog](#)

Infrastructure

- [Google Cloud Blog](#)
- [AWS Blog](#)
- [Azure Architecture Center](#)

Ride-sharing

- [Uber Engineering](#)
- [Lyft Engineering](#)

E-commerce

- [Amazon Science](#)
- [eBay Tech Blog](#)

Paper Reading List (Advanced)

1. [Amazon DynamoDB - Key-Value Store](#)
2. [Google Spanner - Global Scale Database](#)
3. [Google MapReduce - Data Processing](#)
4. [Google BigTable - Structured Storage](#)
5. [Kafka - A Distributed Messaging System](#)
6. [ZooKeeper - Coordination Service](#)
7. [Raft - Consensus Algorithm](#)
8. [The Google File System](#)

Key Concepts Checklist

Must Know

- ☐ Scaling (vertical vs horizontal)
- ☐ Load balancing
- ☐ Caching (all strategies)
- ☐ Message queues
- ☐ SQL vs NoSQL
- ☐ Sharding/partitioning
- ☐ CAP theorem & PACELC
- ☐ Consistent hashing
- ☐ Rate limiting
- ☐ CDNs
- ☐ DNS
- ☐ API design (REST vs gRPC)

Should Know

- ☐ Consensus (Raft, Paxos)
- ☐ Event sourcing & CQRS
- ☐ Two-phase commit
- ☐ Heartbeat & election
- ☐ Distributed caching
- ☐ Blue-green deployment
- ☐ Circuit breakers
- ☐ Idempotency
- ☐ Backpressure
- ☐ Chaos engineering

Nice to Know

- ☐ CRDTs

- ☐ Vector clocks
- ☐ Consistent cutting
- ☐ Change data capture
- ☐ Partition tolerance in practice
- ☐ Metrics, logging, tracing (observability)

06. Core Concepts

System Design Fundamentals

System Design Fundamentals

What is System Design?

System design is the process of defining the architecture, components, modules, interfaces, and data flow of a system to satisfy specified requirements. It involves making high-level decisions about the structure and behavior of a system.

Key Concepts

1. Scalability

The ability of a system to handle increased load by adding resources.

Types:

- **Vertical Scaling (Scale Up):** Adding more power to existing machines
- **Horizontal Scaling (Scale Out):** Adding more machines

2. Reliability

The probability that a system will correctly perform its required function under stated conditions for a specified period.

Key Metrics:

- Mean Time Between Failures (MTBF)
- Mean Time To Recovery (MTTR)
- Availability = $MTBF / (MTBF + MTTR)$

3. Availability

The degree to which a system is operational and accessible when required for use.

Availability Tiers:

- 99% = 3.65 days downtime/year
- 99.9% = 8.76 hours downtime/year
- 99.99% = 52.56 minutes downtime/year
- 99.999% = 5.26 minutes downtime/year

4. Maintainability

The ease with which a system can be modified to correct faults, improve performance, or adapt to a changed environment.

Factors:

- Modularity
- Composability
- Readability
- Testability

5. Efficiency

How well a system uses its resources to achieve its goals.

Metrics:

- Latency (response time)
- Throughput (requests per second)
- Resource utilization (CPU, memory, disk, network)

System Design Process

1. Requirements Gathering

- Functional requirements
- Non-functional requirements
- Constraints

2. Estimation

- Traffic estimation
- Storage estimation
- Bandwidth estimation

3. High-Level Design

- Identify main components
- Define interfaces
- Data flow

4. Detailed Design

- Database schema
- API design
- Algorithm selection

5. Trade-off Analysis

- Consistency vs Availability
- Latency vs Throughput
- Cost vs Performance

Resources

- [System Design Primer](#)
- [ByteByteGo System Design](#)
- [MIT 6.824 Distributed Systems](#)
- [System Design Interview by Alex Xu](#)

Fundamentals - Detailed Notes

Fundamentals - Detailed Notes

System Design Interview Framework

Step 1: Requirements Clarification (2-3 minutes)

- Ask clarifying questions about functional requirements
- Understand scale: users, data, traffic
- Identify constraints: budget, timeline, team size

Example Questions:

- What is the expected number of users?
- What are the read/write ratios?
- What are the latency requirements?
- Do we need to support real-time updates?

Step 2: Estimation (3-5 minutes)

- Estimate QPS (Queries Per Second)
- Estimate storage requirements
- Estimate bandwidth needs

Formulas:

Daily Active Users (DAU) = Total Users * Daily Usage Rate

QPS = DAU * Avg Requests per User / 86400

Storage = Daily Write * Retention Days * Avg Record Size

Step 3: High-Level Design (10-15 minutes)

- Draw main components
- Define data flow
- Identify bottlenecks

Common Components:

- Load Balancer
- Application Servers
- Database (SQL/NoSQL)
- Cache
- Message Queue
- CDN

Step 4: Detailed Design (15-20 minutes)

- Database schema design
- API endpoint design
- Algorithm selection

Database Design Considerations:

- Normalization vs Denormalization
- Indexing Strategy
- Partitioning Strategy

Step 5: Trade-offs (5-10 minutes)

- Discuss alternatives
- Explain why you chose certain components
- Address potential issues

Key Design Principles

1. Keep It Simple (KISS)

- Don't over-engineer
- Use proven solutions
- Avoid premature optimization

2. Separation of Concerns

- Single Responsibility Principle
- Modular design
- Clear interfaces

3. Defense in Depth

- Multiple layers of security
- Fail-safe defaults
- Least privilege

4. Design for Failure

- Assume components will fail
- Implement redundancy
- Plan for recovery

Common Pitfalls

1. **Jumping to solutions** before understanding requirements
2. **Ignoring non-functional requirements** (performance, scalability)
3. **Not considering failure modes**
4. **Over-engineering** simple problems
5. **Underestimating** data growth

Networking & Protocols

Networking & Protocols

OSI Model (7 Layers)

1. **Physical** - Hardware, cables, signals
2. **Data Link** - MAC addresses, frames
3. **Network** - IP addresses, routing
4. **Transport** - TCP/UDP, ports
5. **Session** - Connection management
6. **Presentation** - Encryption, compression
7. **Application** - HTTP, FTP, SMTP

TCP/IP Model (4 Layers)

1. **Network Interface** - Ethernet, WiFi
2. **Internet** - IP, ICMP
3. **Transport** - TCP, UDP
4. **Application** - HTTP, DNS, SMTP

Key Protocols

HTTP/HTTPS

- Request-Response model
- Methods: GET, POST, PUT, DELETE, PATCH
- Status codes: 2xx, 3xx, 4xx, 5xx
- Stateless, but sessions/cookies maintain state

WebSocket

- Full-duplex communication
- Persistent connection
- Real-time updates (chat, gaming)

gRPC

- Remote Procedure Call framework
- Uses Protocol Buffers (protobuf)
- HTTP/2 based
- Bidirectional streaming

DNS

- Domain Name System
- Translates domain names to IP addresses
- Hierarchical structure: Root → TLD → Authoritative

Load Balancing Algorithms

Round Robin

- Distributes requests sequentially
- Simple but doesn't consider server load

Least Connections

- Routes to server with fewest connections
- Better for varying request durations

IP Hash

- Routes based on client IP
- Ensures session persistence

CDN (Content Delivery Network)

- Distributed cache of static content
- Reduces latency by serving from edge locations
- Examples: Cloudflare, AWS CloudFront

Resources

- [Computer Networking: A Top-Down Approach](#)

- [Cloudflare Learning Center](#)
- [MDN Web Docs - HTTP](#)

Networking - Detailed Notes

Networking - Detailed Notes

TCP vs UDP

TCP (Transmission Control Protocol)

- Reliable delivery
- Ordered packets
- Flow control
- Congestion control
- Use cases: HTTP, FTP, Email

UDP (User Datagram Protocol)

- Unreliable delivery
- No ordering guarantee
- No flow control
- Lower overhead
- Use cases: DNS, Video streaming, Gaming

HTTP/2 vs HTTP/3

HTTP/2

- Multiplexing (multiple streams over single connection)
- Header compression (HPACK)
- Server push
- Binary framing

HTTP/3

- Uses QUIC protocol (UDP-based)
- 0-RTT connection setup
- Improved loss recovery
- Connection migration

DNS Resolution Process

1. Browser cache
2. OS cache
3. Router cache
4. ISP DNS server
5. Root DNS server
6. TLD DNS server
7. Authoritative DNS server

TLS Handshake

1. Client Hello (supported ciphers)
2. Server Hello (chosen cipher)
3. Certificate exchange
4. Key exchange
5. Encrypted communication begins

API Design Patterns

REST

- Resource-based URLs
- HTTP methods for CRUD
- Stateless
- HATEOAS (optional)

GraphQL

- Single endpoint
- Client specifies required data
- Strongly typed schema
- Reduces over-fetching

gRPC

- Protocol Buffers
- HTTP/2 transport
- Streaming support
- Code generation

Resources

- [High Performance Browser Networking](#)
- [RFC 7540 - HTTP/2](#)

- [RFC 9000 - QUIC](#)

Databases & Storage

Databases & Storage

SQL vs NoSQL

SQL (Relational)

- Structured data with schemas
- ACID transactions
- Complex queries with JOINS
- Vertical scaling
- Examples: PostgreSQL, MySQL, Oracle

NoSQL (Non-relational)

- Flexible schemas
- Horizontal scaling
- Eventually consistent
- Examples: MongoDB, Redis, Cassandra, DynamoDB

ACID Properties

1. **Atomicity** - All or nothing transactions
2. **Consistency** - Data remains valid after transactions
3. **Isolation** - Concurrent transactions don't interfere
4. **Durability** - Committed data survives failures

Database Scaling

Vertical Scaling

- Add more CPU/RAM to single server
- Simple but has limits
- Single point of failure

Horizontal Scaling

- Add more servers
- More complex but scalable
- Requires data distribution

Sharding Strategies

Hash-based Sharding

- Use hash function on shard key
- Even distribution
- Hard to add/remove shards

Range-based Sharding

- Partition by value ranges
- Easy to query ranges
- Can create hotspots

Directory-based Sharding

- Lookup table for shard mapping
- Flexible but adds overhead

Replication

Master-Slave

- Single write master
- Multiple read replicas
- Async or sync replication

Multi-Master

- Multiple write masters
- Conflict resolution needed
- Higher availability

Indexing

B-Tree Index

- Balanced tree structure
- Good for range queries
- $O(\log n)$ lookup

Hash Index

- Hash table structure
- Excellent for point queries
- $O(1)$ lookup

Composite Index

- Index on multiple columns
- Can cover queries
- Order matters

Resources

- [Database Internals by Alex Petrov](#)
- [PostgreSQL Documentation](#)
- [MongoDB University](#)
- [Designing Data-Intensive Applications by Martin Kleppmann](#)

Databases - Detailed Notes

Databases - Detailed Notes

Database Selection Guide

Use PostgreSQL when:

- Complex queries and joins needed
- ACID compliance required
- Structured data with relationships
- Need for advanced indexing

Use MongoDB when:

- Flexible schema required
- Document-based storage fits
- Horizontal scaling needed
- Rapid development

Use Redis when:

- High-performance caching
- Session storage
- Real-time leaderboards
- Pub/Sub messaging

Use Cassandra when:

- Write-heavy workloads
- Linear scalability needed

- High availability required
- Time-series data

Use DynamoDB when:

- Serverless architecture
- Predictable performance
- Auto-scaling needed
- AWS ecosystem integration

Data Modeling Patterns

Single Table Design (DynamoDB)

- All data in one table
- Composite primary key
- Minimize round trips
- Denormalization

Embedded Documents (MongoDB)

- Nested documents
- Reduces JOINS
- Data locality
- May lead to duplication

Referenced Documents

- Normalized data
- Uses references/IDs
- Consistency over performance
- More complex queries

Consistency Models

Strong Consistency

- Read after write guarantee
- Higher latency
- Lower availability

Eventual Consistency

- Data will be consistent eventually
- Lower latency

- Higher availability

Causal Consistency

- Causally related operations in order
- Balance between strong and eventual
- Requires tracking dependencies

Query Optimization

Indexing

- Create indexes on frequently queried columns
- Avoid over-indexing (write performance)
- Use composite indexes wisely

Query Analysis

- Use EXPLAIN to analyze queries
- Avoid SELECT *
- Use pagination (LIMIT/OFFSET)

Caching

- Cache frequent queries
- Use materialized views
- Implement query result caching

Resources

- [Designing Data-Intensive Applications](#)
- [Use The Index, Luke](#)
- [PostgreSQL Wiki](#)
- [MongoDB Best Practices](#)

Caching Strategies

Caching Strategies

What is Caching?

Caching is storing frequently accessed data in a temporary storage layer to reduce latency and database load.

Caching Patterns

1. Cache-Aside (Lazy Loading)

- Application checks cache first
- If miss, fetch from database
- Store result in cache

Pros:

- Only requested data is cached
- Cache failures don't affect database

Cons:

- Cold start (first request always hits DB)
- Cache stampede possible

2. Write-Through

- Write to cache and database simultaneously
- Cache always has fresh data

Pros:

- Strong consistency
- No cache misses after writes

Cons:

- Higher write latency
- May cache unused data

3. Write-Behind (Write-Back)

- Write to cache first
- Asynchronously write to database

Pros:

- High write performance
- Can batch database writes

Cons:

- Data loss risk on failure
- Complexity in implementation

4. Read-Through

- Cache layer handles database reads
- Application only talks to cache

Pros:

- Simplified application code
- Automatic cache population

Cons:

- Cache layer complexity
- First read still slow

Cache Invalidation

Time-based (TTL)

- Expire after fixed time
- Simple to implement
- May serve stale data

Event-based

- Invalidate on data change
- More complex but accurate
- Requires event tracking

Version-based

- Version cache entries
- Can rollback if needed
- More storage overhead

Cache Topology

Local Cache

- In-process cache (e.g., LRU)
- Very fast (nanoseconds)
- Not shared across instances

Distributed Cache

- Separate cache service (e.g., Redis)
- Shared across instances
- Network latency (milliseconds)

Multi-level Cache

- L1: Local cache

- L2: Distributed cache
- L3: Database

Popular Cache Solutions

Redis

- In-memory data structure store
- Supports strings, hashes, lists, sets
- Persistence options
- Pub/Sub messaging

Memcached

- Simple key-value store
- Multi-threaded
- No persistence
- Great for simple caching

Resources

- [Redis Documentation](#)
- [Caching Patterns](#)
- [High Performance Browser Networking](#)
- [Caching at Reddit](#)

Caching - Detailed Notes

Caching - Detailed Notes

Cache-Stampede Problem

When a popular cache entry expires, many requests hit the database simultaneously.

Solutions:

1. **Locking:** Only one request refreshes cache
2. **Early Expiration:** Refresh before actual expiration
3. **Probabilistic Refresh:** Random requests refresh cache

Cache Penetration

When queries for non-existent data bypass cache and hit database.

Solutions:

1. **Bloom Filter:** Check if key exists before querying
2. **Null Object Caching:** Cache empty results briefly

Cache Avalanche

When multiple cache entries expire simultaneously.

Solutions:

1. **Staggered TTLs:** Add randomness to expiration
2. **Circuit Breaker:** Stop cache requests on failure
3. **Warm Cache:** Pre-populate cache during low traffic

Distributed Cache Strategies

Consistent Hashing

- Distribute keys across cache nodes
- Minimize redistribution on node changes

Replication

- Replicate cache across multiple nodes
- Read from replicas for availability

Partitioning

- Split cache across multiple nodes
- Each node handles subset of keys

Monitoring Cache Performance

Key Metrics:

- **Hit Rate:** $\text{Cache hits} / (\text{hits} + \text{misses})$
- **Eviction Rate:** $\text{Entries removed} / \text{total entries}$
- **Memory Usage:** $\text{Current} / \text{Maximum}$

Target Hit Rates:

- | 95% for hot data
- | 80% for warm data
- | 50% for cold data

Redis Best Practices

Memory Management

- Set maxmemory policy
- Use appropriate data structures
- Monitor memory usage

Persistence

- RDB snapshots for backups
- AOF for durability
- Choose based on use case

Cluster Mode

- Shard data across nodes
- Automatic failover
- Horizontal scaling

Load Balancing & Proxies

Load Balancing & Proxies

What is Load Balancing?

Load balancing distributes incoming network traffic across multiple servers to ensure no single server bears too much demand.

Load Balancing Algorithms

Round Robin

- Distributes requests sequentially
- Simple and predictable
- Doesn't consider server load

Weighted Round Robin

- Servers assigned weights
- Higher weight = more requests
- Good for heterogeneous servers

Least Connections

- Routes to server with fewest active connections
- Adapts to varying request durations
- Better for long-lived connections

IP Hash

- Uses client IP to determine server
- Ensures session persistence
- May cause uneven distribution

Least Response Time

- Routes to fastest responding server
- Considers both load and latency
- Requires health monitoring

Types of Load Balancers

L4 (Transport Layer)

- Works at TCP/UDP level
- Faster (less inspection)
- Based on IP + Port
- Examples: AWS NLB, HAProxy (TCP mode)

L7 (Application Layer)

- Works at HTTP/HTTPS level
- Content-based routing
- SSL termination
- Examples: AWS ALB, Nginx, HAProxy

Proxy Types

Forward Proxy

- Sits in front of clients
- Anonymizes client requests
- Content filtering
- Example: Corporate proxy

Reverse Proxy

- Sits in front of servers
- Load balancing

- SSL termination
- Caching
- Example: Nginx, HAProxy

Health Checks

Active Health Checks

- Periodic probe requests
- Detect failures quickly
- May cause overhead

Passive Health Checks

- Monitor actual traffic
- No extra overhead
- Slower failure detection

Popular Load Balancers

Nginx

- High performance
- Reverse proxy + load balancer
- SSL termination
- Caching

HAProxy

- High availability
- TCP and HTTP balancing
- Advanced health checks
- Stats dashboard

AWS ALB/NLB

- Managed service
- Auto-scaling
- Integration with AWS ecosystem
- Pay per use

Resources

- [Nginx Documentation](#)

- [HAProxy Documentation](#)
- [AWS ELB Documentation](#)
- [Load Balancing Algorithms](#)

Load Balancing - Detailed Notes

Load Balancing - Detailed Notes

SSL Termination

Load balancer handles SSL/TLS encryption/decryption, reducing backend server overhead.

Benefits:

- Reduced backend CPU usage
- Centralized certificate management
- SSL session reuse

Considerations:

- Backend traffic may need encryption
- Compliance requirements
- End-to-end encryption needs

Session Persistence (Sticky Sessions)

Methods:

1. **IP Hash:** Route based on client IP
2. **Cookie-based:** Insert cookie for session tracking
3. **URL-based:** Include session ID in URL

Trade-offs:

- **Pros:** Simple, maintains state
- **Cons:** Uneven load, failover issues

High Availability Patterns

Active-Passive

- Primary handles traffic
- Standby takes over on failure
- Simple but underutilizes resources

Active-Active

- All servers handle traffic
- Better resource utilization
- More complex configuration

Auto-scaling Integration

Horizontal Pod Autoscaler (Kubernetes)

- Scale based on CPU/memory
- Custom metrics support
- Cool-down periods

AWS Auto Scaling

- Scale based on CloudWatch metrics
- Predictive scaling
- Instance refresh

Common Configurations

Nginx Load Balancer

```
upstream backend {  
    server backend1.example.com;  
    server backend2.example.com;  
    server backend3.example.com;  
}  
  
server {  
    listen 80;  
    location / {  
        proxy_pass http://backend;  
    }  
}
```

HAProxy Configuration

```
backend web_servers
    balance roundrobin
    server web1 192.168.1.10:80 check
    server web2 192.168.1.11:80 check
    server web3 192.168.1.12:80 check
```

Resources

- [Nginx Load Balancing](#)
- [HAProxy Configuration Guide](#)
- [AWS ELB Best Practices](#)

Message Queues & Streaming

Message Queues & Streaming

What are Message Queues?

Message queues enable asynchronous communication between components by storing messages in a queue until they are processed.

Key Concepts

Producer

- Creates and sends messages
- Doesn't wait for processing
- Can send to multiple queues

Consumer

- Receives and processes messages
- Can run independently
- Can scale horizontally

Queue

- Stores messages temporarily
- Ensures message delivery
- Handles ordering and durability

Message Queue Patterns

Point-to-Point

- One message, one consumer
- Simple task distribution
- Example: Job queues

Publish-Subscribe

- One message, multiple consumers
- Event broadcasting
- Example: News feeds

Request-Reply

- Send request, wait for reply
- Correlation ID for matching
- Example: RPC calls

Popular Message Queues

Apache Kafka

- Distributed streaming platform
- High throughput
- Horizontal scaling
- Event sourcing support

RabbitMQ

- AMQP protocol
- Flexible routing
- Message acknowledgment
- Easy to use

Amazon SQS

- Fully managed service
- Standard and FIFO queues
- Auto-scaling
- Pay per message

Redis Pub/Sub

- In-memory messaging
- Low latency

- Simple to implement
- No persistence by default

Message Queue Use Cases

1. **Decoupling Services:** Reduce dependencies
2. **Load Leveling:** Handle traffic spikes
3. **Asynchronous Processing:** Background jobs
4. **Event-Driven Architecture:** React to events
5. **Data Pipeline:** ETL processes

Resources

- [Apache Kafka Documentation](#)
- [RabbitMQ Tutorials](#)
- [Designing Event-Driven Systems](#)
- [AWS SQS Documentation](#)

Message Queues - Detailed Notes

Message Queues - Detailed Notes

Kafka vs RabbitMQ

Apache Kafka

- **Architecture:** Distributed log
- **Throughput:** Millions of messages/sec
- **Storage:** Persistent log
- **Ordering:** Per partition
- **Use Cases:** Event streaming, log aggregation, data pipeline

RabbitMQ

- **Architecture:** Message broker
- **Throughput:** Thousands of messages/sec
- **Storage:** In-memory (with persistence)
- **Ordering:** Per queue
- **Use Cases:** Task queues, RPC, complex routing

Event Sourcing

Store all changes as a sequence of events instead of current state.

Benefits:

- Complete audit trail
- Temporal queries
- Event replay
- Decoupled services

Challenges:

- Event versioning
- Schema evolution
- Query complexity

CQRS (Command Query Responsibility Segregation)

Separate read and write models for better scalability.

Benefits:

- Independent scaling
- Optimized data models
- Better performance

Challenges:

- Eventual consistency
- Complexity
- Data synchronization

Message Ordering

Strict Ordering

- Single partition/queue
- Sequential processing
- Lower throughput

Partial Ordering

- Partition by key
- Order within partition
- Higher throughput

No Ordering

- Independent consumers
- Maximum throughput
- Simpler implementation

Dead Letter Queues

Store messages that fail processing after max retries.

Benefits:

- Prevent message loss
- Enable debugging
- Separate error handling

Implementation:

- Configure retry count
- Move to DLQ on failure
- Monitor and alert

Exactly-Once Delivery

Challenges:

- Network failures
- Consumer crashes
- Duplicate messages

Solutions:

- Idempotent consumers
- Transactional outbox
- Kafka transactions
- Deduplication

Resources

- [Kafka vs RabbitMQ](#)
- [Event Sourcing Pattern](#)
- [CQRS Pattern](#)

Consistent Hashing & Partitioning

Consistent Hashing & Partitioning

What is Consistent Hashing?

Consistent hashing is a technique that distributes data across nodes in a way that minimizes redistribution when nodes are added or removed.

The Problem

With simple hashing ($\text{key} \% N$):

- Adding/removing a node redistributes almost all keys
- Causes massive data movement
- Degrades performance during scaling

How Consistent Hashing Works

1. **Hash Ring:** Nodes and keys are mapped to positions on a ring
2. **Assignment:** Key is assigned to next node clockwise
3. **Minimal Redistribution:** Only keys between old and new node move

Virtual Nodes

Physical nodes are mapped to multiple positions on the ring.

Benefits:

- Better distribution
- Heterogeneous hardware support
- Easier scaling

Configuration:

- More vnodes = better balance
- More memory for lookup tables
- Typical: 100-200 vnodes per node

Partitioning Strategies

Hash-based Partitioning

- Hash the partition key
- Even distribution
- Range queries expensive

Range-based Partitioning

- Partition by value ranges

- Good for range queries
- Risk of hotspots

Directory-based Partitioning

- Lookup table for mapping
- Flexible redistribution
- Single point of failure

Use Cases

1. **Distributed Caches:** Redis Cluster, Memcached
2. **Distributed Databases:** Cassandra, DynamoDB
3. **Load Balancers:** Session affinity
4. **CDNs:** Content distribution

Implementation Example

```
import hashlib
from bisect import bisect_right

class ConsistentHash:
    def __init__(self, nodes, vnodes=100):
        self.ring = {}
        self.sorted_keys = []
        for node in nodes:
            for i in range(vnodes):
                key = self._hash(f"{node}:{i}")
                self.ring[key] = node
                self.sorted_keys.append(key)
            self.sorted_keys.sort()

    def _hash(self, key):
        return int(hashlib.md5(key.encode()).hexdigest(), 16)

    def get_node(self, key):
        if not self.ring:
            return None
        h = self._hash(key)
        idx = bisect_right(self.sorted_keys, h) % len(self.sorted_keys)
        return self.ring[self.sorted_keys[idx]]
```

Resources

- [Consistent Hashing - Wikipedia](#)
- [Dynamo Paper \(Amazon\)](#)
- [MIT 6.824 Labs](#)
- [Consistent Hashing and Random Trees](#)

Consistent Hashing - Detailed Notes

Consistent Hashing - Detailed Notes

Advanced Topics

Bounded Loads

- Limit load per node
- Prevent hotspots
- Better utilization

Weighted Consistent Hashing

- Nodes have different capacities
- Weight affects ring position
- Supports heterogeneous hardware

Jump Hash

- Simple and fast
- Good distribution
- No virtual nodes needed
- Limitation: only supports adding nodes

Comparison of Partitioning Methods

Method	Pros	Cons	Best For
Hash-based	Even distribution	Range queries hard	Key-value stores
Range-based	Range queries easy	Hotspots possible	Time-series data
Directory-based	Flexible	Single point of failure	Dynamic environments
Consistent Hashing	Minimal redistribution	Complexity	Distributed caches

Real-World Implementations

Redis Cluster

- 16384 hash slots
- Each node responsible for slots
- Supports partial resharding

Cassandra

- Token ring with vnodes
- Configurable replication
- Automatic load balancing

Amazon DynamoDB

- Consistent hashing
- Virtual nodes
- Replication across AZs

Handling Node Failures

Detection

- Heartbeat monitoring
- Gossip protocol
- Timeout-based

Recovery

- Replication ensures availability
- Hinted handoff for writes
- Read repair for consistency

Resources

- [Dynamo Paper](#)
- [Consistent Hashing with Bounded Loads](#)
- [Jump Hash](#)

CAP Theorem & Consistency

CAP Theorem & Consistency

CAP Theorem

In a distributed system, you can only guarantee two of three properties simultaneously:

1. **Consistency:** Every read receives the most recent write
2. **Availability:** Every request receives a response
3. **Partition Tolerance:** System continues despite network failures

CAP Combinations

CP (Consistency + Partition Tolerance)

- Consistent but may reject requests
- Examples: MongoDB, HBase, Redis Cluster
- Use when: Strong consistency required

AP (Availability + Partition Tolerance)

- Available but may return stale data
- Examples: Cassandra, DynamoDB, CouchDB
- Use when: High availability required

CA (Consistency + Availability)

- No partition tolerance
- Only for single-node systems
- Not practical for distributed systems

PACELC Theorem

Extension of CAP: In case of **P**artition, choose **A**vailability or **C**onsistency; **E**lse, choose **L**atency or **C**onsistency.

PACELC Combinations:

- **PA/EL:** Cassandra, DynamoDB
- **PA/EC:** MongoDB, HBase
- **PC/EL:** -
- **PC/EC:** Spanner, Percolator

Consistency Models

Strong Consistency

- Linearizable reads
- All replicas see same data
- Higher latency

- Examples: Spanner, Zookeeper

Eventual Consistency

- Replicas converge eventually
- May read stale data
- Lower latency
- Examples: Cassandra, DynamoDB

Causal Consistency

- Causally related operations ordered
- Non-causal operations may be concurrent
- Balance between strong and eventual
- Examples: MongoDB, Riak

Read-Your-Writes Consistency

- User sees their own writes immediately
- Others may see stale data
- Common in user-facing applications

Quorum-based Consistency

Quorum Formula

- $W + R > N$ (for consistency)
- W: Write quorum
- R: Read quorum
- N: Replication factor

Tunable Consistency

- $W=1, R=N$: Fast writes, slow reads
- $W=N, R=1$: Slow writes, fast reads
- $W=Q, R=Q$: Balanced

Conflict Resolution

Last-Write-Wins (LWW)

- Simple but may lose data
- Uses timestamps
- No conflict detection

Vector Clocks

- Track causality
- Detect conflicts
- More complex

Application-level Resolution

- Custom logic
- Domain-specific rules
- Most flexible

Resources

- [CAP Theorem - Brewer's Keynote](#)
- [Raft Paper](#)
- [Jepsen Consistency Tests](#)
- [Please stop calling databases CP or AP](#)

CAP Theorem - Detailed Notes

CAP Theorem - Detailed Notes

Consensus Algorithms

Raft

- Leader-based consensus
- Log replication
- Leader election
- Safety guarantees
- Used in: etcd, Consul

Paxos

- Classical consensus algorithm
- Complex to implement
- Used in: Google Chubby, Zookeeper
- Variants: Multi-Paxos, Fast Paxos

Practical Implications

When to Choose CP

- Financial transactions
- Inventory management
- User authentication
- Any system requiring strong consistency

When to Choose AP

- Social media feeds
- DNS resolution
- Shopping carts
- Any system requiring high availability

Consistency in Practice

Read-after-Write Consistency

- User reads their own writes
- Implementation: Read from primary after write

Monotonic Reads

- Once you read a value, you won't see older values
- Implementation: Track version per client

Consistent Prefix Reads

- Reads see writes in correct order
- Implementation: Track causal dependencies

Data Replication Strategies

Synchronous Replication

- Write to all replicas before success
- Strong consistency
- Higher latency

Asynchronous Replication

- Write to primary, replicate later
- Eventual consistency
- Lower latency

Semi-synchronous Replication

- Write to primary + one replica

- Balance of consistency and latency

Failure Modes

Network Partition

- Nodes can't communicate
- Must choose availability or consistency
- CAP theorem applies

Node Failure

- Single node goes down
- Replicas take over
- No CAP trade-off needed

Byzantine Failure

- Node behaves arbitrarily
- Requires Byzantine fault tolerance
- Complex and expensive

Resources

- [Raft Consensus Algorithm](#)
- [Paxos Made Simple](#)
- [Understanding CAP Theorem](#)
- [Consistency Models in Distributed Systems](#)

07. Guided Build Projects

Chat System - System Design

Chat System - System Design

Requirements

Functional Requirements

- One-on-one and group chat (up to 1,000 members)
- Real-time message delivery
- Message history
- Presence (online/offline) status
- Read receipts
- Typing indicators
- Media sharing (images, files)
- Message search (optional)

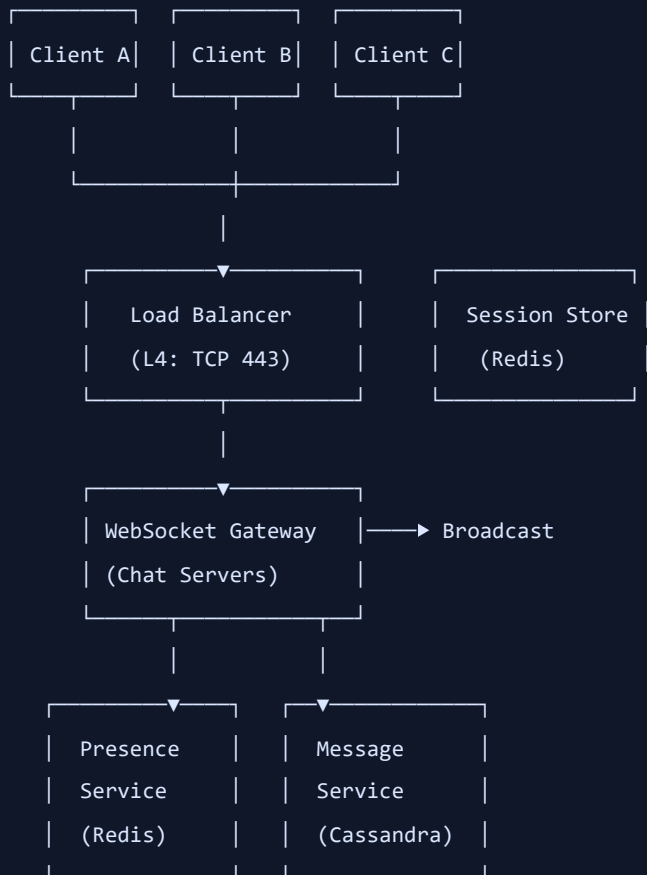
Non-Functional Requirements

- End-to-end latency < 100ms
- Availability > 99.9%
- Messages never lost (durable)
- Handle millions of concurrent connections
- Message ordering per conversation

Capacity Estimation

- **MAU:** 100M
- **Concurrent connections:** 5M (5% of MAU)
- **Messages/day:** 5B (50 msgs/user/day)
- **Message size:** ~100 bytes text
- **Storage:** 5B/day * 100B = 500GB/day text + images + files

High-Level Design



WebSocket Server Flow

1. Client connects: `wss://chat.example.com/ws?token=...`
2. Gateway authenticates (validate JWT)
3. Gateway registers connection in Session Store
4. Gateway subscribes to conversation channels (Redis Pub/Sub)
5. Client sends: `{type: "message", conv: "abc", text: "hello"}`
6. Gateway:
 - a. Appends to conversation feed (Cassandra / Kafka)
 - b. Publishes to Redis channel ``conv:abc``
 - c. Subscribed instances deliver to connected clients

Message Flow (Send → Receive)

Message → Enqueue to Kafka (durability, ordering per partition)



Partition by conversation_id (ordering)



Message State Store (Cassandra) - history



Publish to all chat servers via Redis Pub/Sub



Deliver to connected clients



If client offline → Push Notification + store offline

Database Schema (Cassandra)

-- Message Table: partitioned by conversation, ordered by time

```
CREATE TABLE messages (  
  message_id TIMEUUID,  
  conversation_id UUID,  
  sender_id BIGINT,  
  type TEXT,          -- text, image, file  
  body TEXT,  
  metadata MAP<TEXT, TEXT>,  
  sent_at TIMESTAMP,  
  PRIMARY KEY (conversation_id, sent_at, message_id)  
) WITH CLUSTERING ORDER BY (sent_at DESC);
```

-- Conversation Membership

```
CREATE TABLE conversation_members (  
  conversation_id UUID,  
  user_id BIGINT,  
  joined_at TIMESTAMP,  
  last_read_message UUID,  
  PRIMARY KEY (conversation_id, user_id)  
);
```

-- Secondary index on user_id for "my conversations"

-- User-Conversation Index (reverse lookup)

```
CREATE TABLE user_conversations (  
  user_id BIGINT,  
  conversation_id UUID,  
  last_activity TIMESTAMP,  
  PRIMARY KEY (user_id, last_activity)  
) WITH CLUSTERING ORDER BY (last_activity DESC);
```

-- Unread Counts (fast counter)

```
CREATE TABLE unread_counts (  
  user_id BIGINT,  
  conversation_id UUID,  
  unread_count COUNTER,  
  PRIMARY KEY (user_id, conversation_id)  
);
```

Presence System (Redis)

Online status per user:

```
SET user:{id}:status "online"  
SET user:{id}:last_seen <timestamp>
```

Connection registration:

```
SADD presence:{userId} {socketId}
```

Broadcast on connect/disconnect:

```
PUBLISH presence:events {"user": 123, "status": "online"}
```

Presence Edge Cases

1. **Multi-device:** Track per device, aggregate
2. **Connection drop:** Heartbeat (ping every 30s)
3. **Status transitions:** Throttle broadcasts (debounce)

Read Receipts

Read event: {type: "read", user: 123, conv: "abc", message_id: "xyz"}

- Update last_read_message in Cassandra
- Unread count = messages after last_read
- Broadcast to other members

Message Ordering

Guarantee: Messages in a conversation arrive in order.

Strategy: **Partition by conversation_id in Kafka**

- Kafka guarantees order per partition
- Message sequence number per conversation
- Client handles gap detection (missing seq)

Message Deduplication

Client may retry on network failure → duplicate messages.

Solution: Client generates message_id (UUID); Server dedupes using `conflict-free` set per conversation (e.g., store seen message_ids with TTL).

Offline Experience

1. Client disconnects
2. Messages stored in Cassandra
3. Push notification sent (title + preview)
4. On reconnect:
 - Client requests messages after last_sync timestamp
 - Batch fetch recent messages
 - Update read receipts

Scaling Strategy

Component	Scale Method
Chat servers	Horizontal (stateless)
Connections	LB with sticky sessions / consistent hash of user_id
Kafka partitions	Increase with conversations
Cassandra	Shard/replicate by conversation_id
Redis	Cluster mode (presence + pub/sub)
Notifications	APNs/FCM pools

Interview Talking Points

1. **Why WebSocket over HTTP long-polling?:** Full-duplex, low overhead, real-time
2. **Why Kafka?:** Durable, ordered (per conversation), replayable
3. **Why Cassandra?:** Write-heavy, time-ordered, no schema rigidity
4. **500M+ users:** Multi-region; Kafka cross-region mirrors; conflict-free chat
5. **Group chat > 1,000:** Fan-out to topic/channel systems (massive like Discord)

Advanced: Discord/Spaces Scale

- Channels vs conversations (many-to-many)
- Voice: WebRTC SFU (Selective Forwarding Unit)
- Moderation: canned bot, rate limiting, profanity filter, ML-based
- GDPR: delete user data on account deletion

Resources

- [Socket.io Documentation](#)
- [PubNub Chat Architecture](#)
- [How Discord Scaled to Millions of Concurrent Users](#)

- [Designing a Multipurpose Chat App](#)

Notification System - System Design

Notification System - System Design

Requirements

Functional Requirements

- Send push notifications, emails, SMS
- Support multiple channels (push, email, SMS, in-app)
- User preferences and opt-outs
- Delivery retries with backoff
- Rate limiting per user/channel
- Notification templates

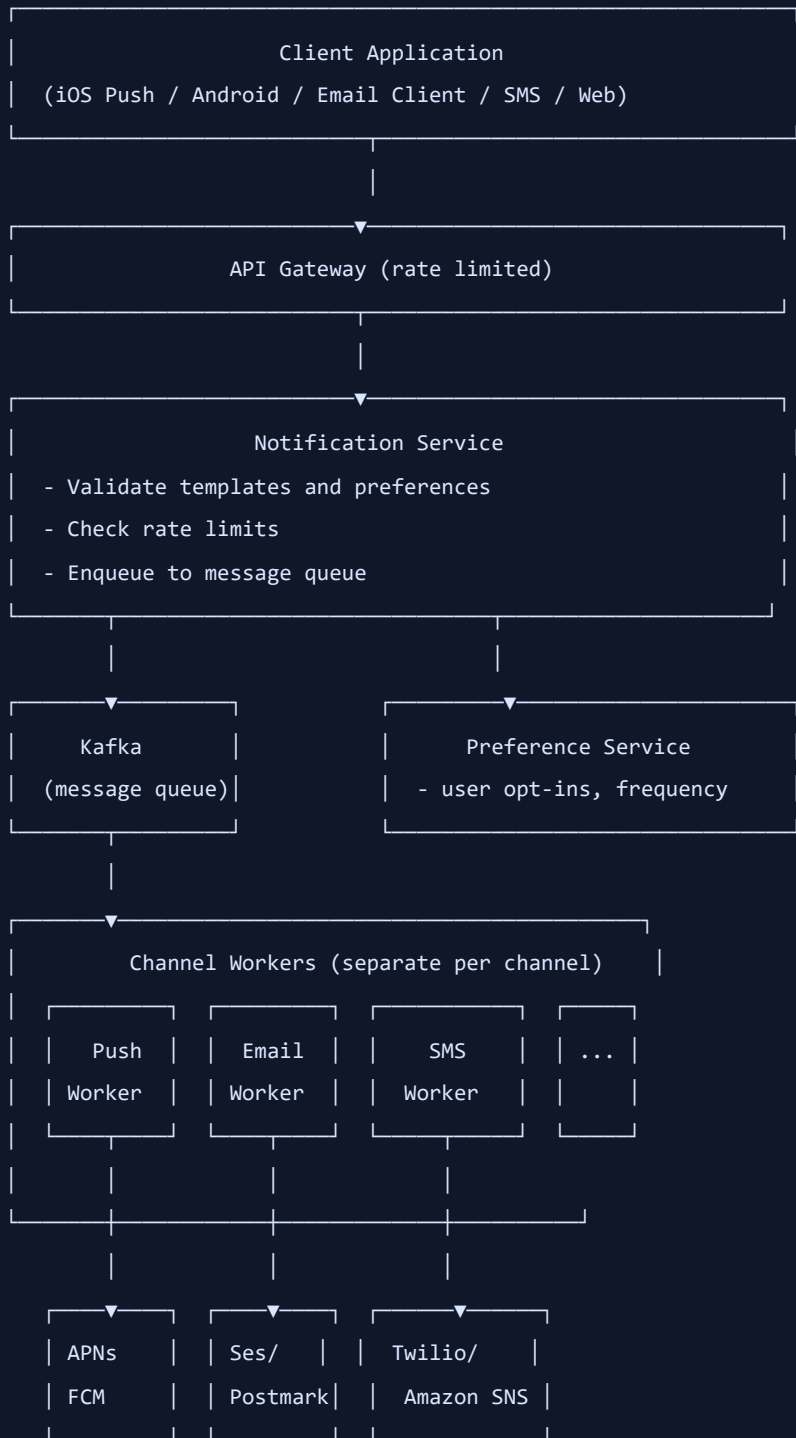
Non-Functional Requirements

- **At-least-once** delivery (no data loss)
- High availability (>99.9%)
- Scalability (100M+ notifications/day)
- Hard latency: push < 1min, email < 5min

Capacity Estimation

- Notifications/day: 100M
- Push: 60%, Email: 30%, SMS: 10%
- Peak QPS: $100M/86400s \approx 1,200$ writes/sec → 3,000 with spikes
- Storage: $100M * 1KB = 100GB/day$

High-Level Design



Notification Flow

1. Client calls POST /notifications

```
{  
  "type": "push",  
  "template": "welcome",  
  "data": {"name": "Alice"},  
  "target": {"user_id": 12345}  
}
```

2. Notification service:

- Fetches user preferences
- Checks rate limit
- Renders template
- Enqueues to Kafka topic

3. Channel worker consumes:

- Push worker → APNs/FCM
- Email worker → SES/SendGrid
- SMS worker → Twilio

4. Track delivery status:

- Update status in DB
- Retry with exponential backoff

Database Schema

```

-- Notifications Table
CREATE TABLE notifications (
    id UUID PRIMARY KEY,
    user_id BIGINT NOT NULL,
    notification_type VARCHAR(50),
    channel VARCHAR(20), -- push, email, sms
    template_id VARCHAR(100),
    data JSONB,
    status VARCHAR(20), -- pending, sent, delivered, failed
    scheduled_at TIMESTAMPTZ,
    sent_at TIMESTAMPTZ,
    delivered_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Partition by created_at (monthly)

-- User Preferences
CREATE TABLE notification_preferences (
    user_id BIGINT PRIMARY KEY,
    push_enabled BOOLEAN DEFAULT TRUE,
    email_enabled BOOLEAN DEFAULT TRUE,
    sms_enabled BOOLEAN DEFAULT FALSE,
    quiet_hours_start TIME,
    quiet_hours_end TIME,
    max_per_hour INT DEFAULT 10
);

-- Templates
CREATE TABLE notification_templates (
    id VARCHAR(100) PRIMARY KEY,
    channel VARCHAR(20),
    subject TEXT,
    body TEXT,
    -- handle airnel variables, locale
    category VARCHAR(50)
);

-- Delivery Logs
CREATE TABLE notification_audit (
    notification_id UUID,
    attempt INT,
    provider VARCHAR(50),

```

```
status VARCHAR(20),
error TEXT,
attempted_at TIMESTAMPTZ,
PRIMARY KEY (notification_id, attempt)
);
```

Key Design Decisions

1. Message Queue (Kafka)

Why:

- Decouple sending from processing
- Buffer traffic spikes
- Multiple consumers (channels)
- Retry capability

2. Per-channel Workers

Why:

- Isolate failures (email down doesn't block push)
- Independent scaling
- Provider-specific rate limits

3. Templates with Variables

Why:

- Consistent formatting
- Reusable + localized
- Dynamic content in fixed structure

4. Retry with Backoff

```
Attempt 1: 0s
Attempt 2: 30s
Attempt 3: 5min
Attempt 4: 30min
Attempt 5: 2hr
→ Dead Letter Queue after 5 attempts
```

Reliability & Failure Handling

Provider Down (APNs/SES/Twilio)

- Retry with backoff
- Route to alternate provider
- Dead letter queue for manual review

Duplicate Delivery

- Send with idempotency key
- Providers support dedup (APNs)
- At-least-once is acceptable for notifications

Rate Limited by Provider

- Per-provider token bucket
- Queue-based pacing
- Bulk operations to reduce calls

Scaling Considerations

1. **Kafka partitions**: Shard by user_id hash
2. **Workers**: Autoscale by queue depth
3. **DB**: Partition by created_at
4. **Preference cache**: Redis for hot users
5. **Rate limiter**: Redis token bucket per user/channel

Extensions to Discuss

1. **Scheduled notifications** (push in 3 hours)
2. **A/B testing templates**
3. **Analytics** (deliverability metrics)
4. **Multi-language support**
5. **Deduplication** (same notif can't be sent twice)

Resources

- [OneSignal Documentation](#)
- [Firebase Cloud Messaging](#)
- [APNs Overview](#)
- [Scott's Notification Architecture \(Uber\)](#)

Rate Limiter - System Design

Rate Limiter - System Design

Requirements

Functional Requirements

- Limit requests per client/IP/user
- Different limits per endpoint
- Return proper HTTP 429 responses
- Support distributed systems

Non-Functional Requirements

- Low latency (<1ms overhead)
- High accuracy
- Memory efficient
- Fault tolerant

Rate Limiting Algorithms

1. Token Bucket

- Tokens added at fixed rate
- Request consumes token
- Burst friendly

```
Bucket size: 10 tokens
Refill rate: 2 tokens/second
Request: Consume 1 token
If no tokens: Reject with 429
```

2. Leaky Bucket

- Requests enter queue
- Processed at fixed rate
- Smooths traffic spikes

3. Fixed Window Counter

- Count requests in time window
- Simple to implement
- Edge case: 2x burst at window boundary

4. Sliding Window Log

- Store timestamp of each request

- Count requests in sliding window
- Memory intensive but accurate

5. Sliding Window Counter

- Hybrid of fixed window and log
- Weighted combination of current and previous window
- Good balance of accuracy and efficiency

Distributed Rate Limiting

Redis Implementation

```
-- Token Bucket in Redis
local key = KEYS[1]
local capacity = tonumber(ARGV[1])
local refill_rate = tonumber(ARGV[2])
local now = tonumber(ARGV[3])

local bucket = redis.call('HMGET', key, 'tokens', 'last_refill')
local tokens = tonumber(bucket[1]) or capacity
local last_refill = tonumber(bucket[2]) or now

-- Refill tokens
local elapsed = now - last_refill
local new_tokens = math.min(capacity, tokens + elapsed * refill_rate)

-- Try to consume
if new_tokens >= 1 then
    new_tokens = new_tokens - 1
    redis.call('HMSET', key, 'tokens', new_tokens, 'last_refill', now)
    redis.call('EXPIRE', key, math.ceil(capacity / refill_rate) * 2)
    return 1
else
    return 0
end
```

API Design

```
Rate Limit Headers:  
X-RateLimit-Limit: 100  
X-RateLimit-Remaining: 95  
X-RateLimit-Reset: 1609459200
```

```
HTTP 429 Response:  
{  
  "error": "rate_limit_exceeded",  
  "message": "Too many requests",  
  "retry_after": 30  
}
```

Implementation Strategy

Application Level

- In-memory token bucket per instance
- Fast but not distributed

Redis Level

- Centralized rate limiting
- Lua scripts for atomicity
- Redis Cluster for scaling

Hybrid Approach

- Local cache for hot paths
- Redis for global limits
- Best of both worlds

Resources

- [Cloudflare Rate Limiting](#)
- [Stripe Rate Limiting](#)
- [System Design Interview - Alex Xu](#)

09-projects/url-shortener/README.md

08. Large-Scale System Designs

URL Shortener Deep Dive

URL Shortener Deep Dive

Complete Design Walkthrough

This document takes you through the full design of a URL shortener at scale.

Requirements Analysis

Functional Requirements

1. **Shorten URLs:** Accept long URL, return short code
2. **Redirect:** Route short URL to original
3. **Custom aliases:** Users choose their own codes
4. **Analytics:** Track clicks, referrers, locations
5. **Expiration:** Optional link expiry
6. **User accounts:** Manage links (optional)

Non-Functional Requirements

1. **Availability** > 99.9%
2. **Redirect latency** < 100ms
3. **Scale:** 100M new URLs/day, 100B clicks/day
4. **No data loss**

Scale Estimation

DAU: 100M

New URLs/day: 10M

URL lookups/day: 100M

QPS (lookup): $100M / 86400 \approx 1,200$ reads/sec $\rightarrow$ peak 3,000

QPS (create): $10M / 86400 \approx 115$ writes/sec $\rightarrow$ peak 300

Storage (5 years):

$10M \text{ new URLs/day} \times 365 \times 5 = 18B \text{ URLs}$

Each URL ~500 bytes = 9TB

Plus indexes $\approx$ 15TB

API Design

```
# Create short URL
POST /api/shorten
{
  "url": "https://example.com/very/long/path",
  "custom_alias": "my-link",          // optional
  "expiry": "2025-12-31T00:00:00Z"    // optional
}
Response:
{
  "short_code": "abc123",
  "short_url": "https://short.ly/abc123",
  "expiry": "2025-12-31T00:00:00Z"
}

# Redirect
GET /abc123
Location: https://example.com/very/long/path
Status: 301 (permanent) or 302 (temporary)

# Get stats
GET /api/stats/abc123
{
  "clicks_24h": 342,
  "clicks_total": 15432,
  "top_referrers": [{"source": "twitter.com", "count": 8921}]
}
```

Short Code Generation

Option 1: Base62 Encoding

- Alphabet: `a-z A-Z 0-9` (62 chars)
- 7 chars: $62^7 \approx 3.5$ trillion URLs
- Generate unique ID → convert to Base62

```
BASE62 = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
```

```
def encode_base62(num):  
    if num == 0:  
        return BASE62[0]  
    result = []  
    while num > 0:  
        num, rem = divmod(num, 62)  
        result.append(BASE62[rem])  
    return ''.join(reversed(result))  
  
def decode_base62(code):  
    return sum(BASE62.index(c) * (62 ** i)  
              for i, c in enumerate(reversed(code)))
```

Option 2: Hash-based

- MD5/SHA256 → take first 6-8 chars
- Collision handling needed

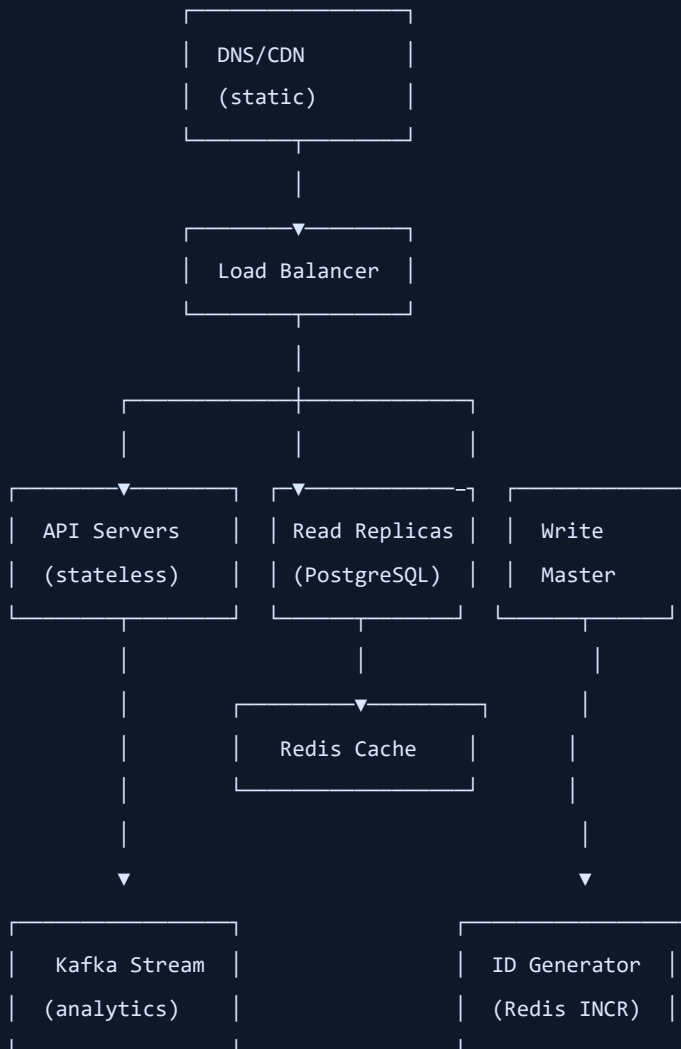
Option 3: Counter-based (Recommended)

```
# Use a distributed unique ID service:  
# 1. Unique ID via Redis INCR or Snowflake  
# 2. Encode to Base62  
id = get_unique_id()      # e.g., 124567890123  
short_code = encode_base62(id)  # "abcd123"
```

Database Schema

```
CREATE TABLE urls (  
    id BIGSERIAL PRIMARY KEY,  
    short_code VARCHAR(10) UNIQUE NOT NULL,  
    original_url TEXT NOT NULL,  
    user_id BIGINT NULL,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    expires_at TIMESTAMPTZ NULL,  
    is_custom BOOLEAN DEFAULT FALSE  
);  
  
-- Indexes  
CREATE INDEX idx_short_code ON urls (short_code);  
CREATE INDEX idx_user_id ON urls (user_id);  
CREATE INDEX idx_expires_at ON urls (expires_at);  
  
-- Click Analytics (separate table or time-series DB)  
CREATE TABLE clicks (  
    job_id BIGSERIAL PRIMARY KEY,  
    short_code VARCHAR(10) NOT NULL,  
    clicked_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    referral TEXT,  
    user_agent TEXT,  
    ip_address INET  
);  
  
-- Consider partitioning by clicked_at
```

Architecture Diagram



Caching Strategy

Heat vs Cold URLs

- 20% of URLs receive 80% of traffic (Pareto)
- Cache hottest 20% in Redis

Cache Flow

```
def redirect(short_code):
    # 1. Try Redis
    url = redis.get(f"url:{short_code}")

    # 2. Cache miss → DB
    if url is None:
        url = db.query("SELECT original_url FROM urls WHERE short_code=%s",
                       short_code)
        if url:
            # Cache with tiered TTL: 1h to 24h
            redis.setex(f"url:{short_code}", 3600, url)

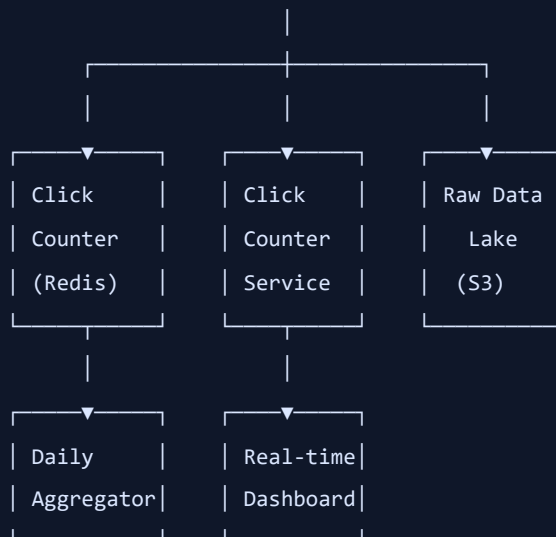
    # 3. Hit rate target >90%
    return event_redirect(url)
```

Cache Stampede Prevention

```
# Use SET NX to prevent stampede
if redis.setnx(f"lock:{short_code}", "1", ex=5):
    url = db_query(...) # only one query
    redis.set(f"url:{short_code}", url, ex=3600)
    redis.delete(f"lock:{short_code}")
```

Analytics Pipeline

Client Redirect → App Server → Kafka (event)



Failure Handling

1. Cache Down

- Falls through to DB
- Use circuit breaker (e.g., Hystrix)
- DB can handle base load

2. DB Down

- Cache serves hot URLs (temporary)
- Write queue builds up
- Failover to replica

3. ID Generator Down

- Redis INCR is redundant (master + replicas)
- Fallback: Snowflake algorithm
- Allocate ID ranges to apps (batching)

Scaling Playbook

At 1M URLs

- Single PostgreSQL + Redis
- 2 app servers

At 100M URLs

- Shard by short_code hash
- Read replicas (3-5)
- Redis Cluster (2-3 nodes)

At 10B URLs

- Global distribution
- Regional deployments
- Multi-master DB
- Edge CDN caching

Interview Talking Points

1. **Why Base62?**: Efficient, URL-safe, compact
2. **Why counter over hash?**: No collisions
3. **301 vs 302**: 301 cacheable for SEO, 302 for analytics

4. **Cache hierarchy:** L1 client → L2 CDN → L3 Redis → DB
5. **Consistency:** Eventually consistent is fine
6. **DB choice:** PostgreSQL for transactional integrity, can shard later

Instagram / Photo Sharing - System Design

Instagram / Photo Sharing - System Design

Requirements

Functional Requirements

- Upload photos/videos
- Feed generation (follow users)
- Likes and comments
- Direct messages
- Stories (24-hour expiring content)
- Search (people, tags, locations)

Non-Functional Requirements

- Feed load latency < 200ms
- Availability > 99.9%
- Media durability (never lose a photo)
- Read-heavy (10:1 read:write)
- Eventually consistent (feed)

Capacity Estimation

- **DAU:** 500M active users
- **Photos/day:** 100M uploads
- **Feed reads/day:** 2B (4 posts/user/day)
- **Photo size:** 500KB avg (compressed)
- **Storage:** 100M * 500KB = 50TB/day new media
- **Feed QPS:** 2B/86400 ≈ 23K QPS reads

High-Level Design

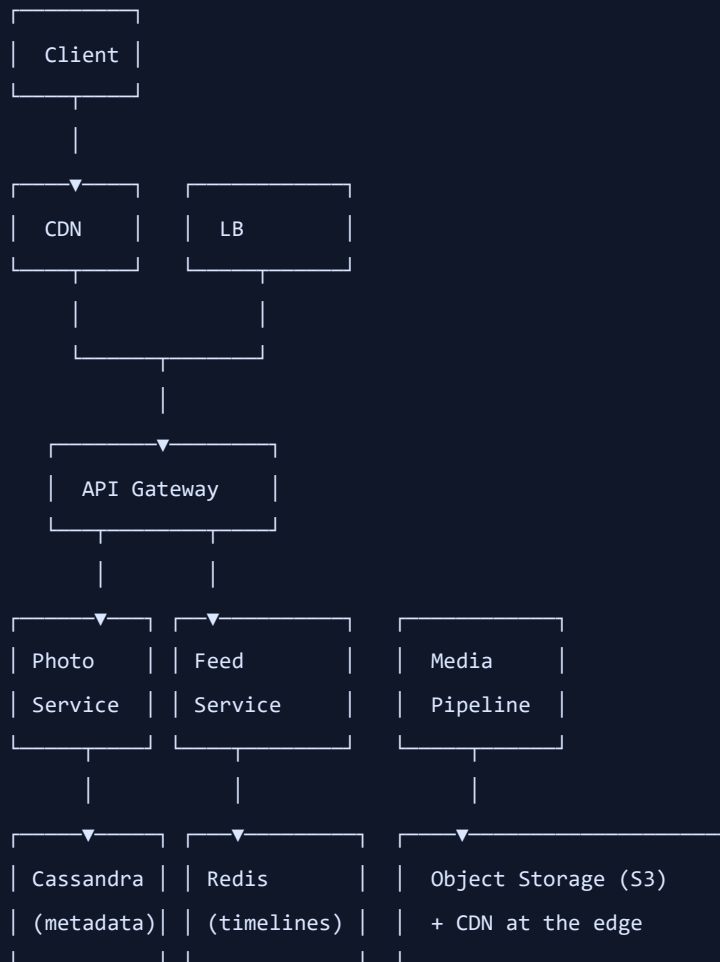


Photo Upload Flow

1. Client requests pre-signed URL from API
2. Client uploads directly to S3 (no server bottleneck)
3. S3 triggers event → Media Pipeline
4. Pipeline:
 - Generate sizes (thumbnail, small, medium, large)
 - Apply compression (WebP/AVIF)
 - Store originals + processed
 - Extract EXIF, generate blurhash
5. Update metadata in Cassandra
6. Notify Feed Service (fan-out to followers)

Media Pipeline Components

S3 Event → Lambda/Worker →

1. Transcode (thumbnail, various resolutions)
2. Optimize (format + quality)
3. Content Moderation (NSFW / image safety ML)
4. Persist to final buckets
5. CDN purge/invalidate old keys

Feed Generation (Hybrid Fanout)

POST /photos { media_id }

- ↳ Log to own timeline
- ↳ Fan-out to follower timelines (Async via queue)

Regular user (≤10K followers):

ZADD timeline:{follower} now media_id // pushed

Celebrity (>10K followers):

Store in celebrity feed (fanout-on-read)

On feed request: merge pulls from celebs

Feed Read

GET /feed?offset=0&limit=20

1. Redis: ZREVRANGE timeline:{user} 0 19
2. Handle pagination
3. For recent celebrities: merge their feeds (capability)
4. Fetch metadata + pre-signed media URLs
5. Serve via API/CDN

Database Schema

```

-- Media metadata (Cassandra)
CREATE TABLE media (
    media_id TIMEUUID,
    user_id BIGINT,
    type TEXT,          -- photo, video, reel
    caption TEXT,
    media_urls MAP<TEXT, TEXT>, -- {thumb, small, medium}
    dimensions MAP<TEXT, INT>,
    location MAP<TEXT, TEXT>,
    created_at TIMESTAMP,
    PRIMARY KEY (user_id, created_at, media_id)
);

-- Follows (Cassandra)
CREATE TABLE follows (
    user_id BIGINT,
    followee_id BIGINT,
    created_at TIMESTAMP,
    PRIMARY KEY (user_id, followee_id)
);

-- Likes (Cassandra - huge volume)
CREATE TABLE likes (
    media_id TIMEUUID,
    liker_id BIGINT,
    created_at TIMESTAMP,
    PRIMARY KEY (media_id, liker_id)
);

-- Comments (Cassandra)
CREATE TABLE comments (
    media_id TIMEUUID,
    comment_id UUID,
    commenter_id BIGINT,
    text TEXT,
    created_at TIMESTAMP,
    PRIMARY KEY (media_id, created_at, comment_id)
);

-- Stories (24h TTL)
CREATE TABLE stories (
    user_id BIGINT,
    story_id UUID,

```

```
media_urls MAP<TEXT, TEXT>,
expires_at TIMESTAMP,
created_at TIMESTAMP,
PRIMARY KEY (user_id, expires_at, story_id)
);
-- Cassandra TTL(expires_at) auto-deletes
```

Caching Strategy

Cache	What	TTL
Redis	Timeline (sorted sets)	30min
Redis	User profile	1hr
Redis	Media metadata	5min
CDN	Media files	30 days (long cache)
CDN	Pre-signed URLs	60s

Note: Media CDN caching is critical - 90% of feed bytes are images.

Analytics & Scaling

Sharding

- Shard by `user_id` for media/follows
- Shard by `media_id` for likes/comments
- `storage:writing content`

Hot Users Problem

- Celebrity posting → thundering herd on followers' timelines
- Solution: cap fanout (only push top-N), else pull merge

Search (Optional)

```
Metadata → Kafka → Elasticsearch index (users, tags, locations, captions)
Query → ES → Return user IDs → Join with media → Serve
```

Interview Talking Points

1. **Why direct-to-S3 upload?** Removes bottleneck, scale-out
2. **Why Cassandra?** Write-heavy with time-ordered data

3. **Hybrid fanout**: push + pull, celebrity lists
4. **CDN-first**: static + media delivery
5. **Stories TTL**: automatic cleanup, no cron jobs
6. **Content moderation**: pipeline before publish

Resources

- [Instagram Engineering](#)
- [A Brief History of Scaling @Ifeng \(Instagram\)](#)
- [Designing Instagram - Educative](#)
- [How Instagram Feeds Work](#)

Twitter/News Feed - System Design

Twitter/News Feed - System Design

Requirements

Functional Requirements

- Post tweets (text, images, videos)
- Follow/unfollow users
- View timeline (home feed)
- View user feed
- Like, retweet, reply

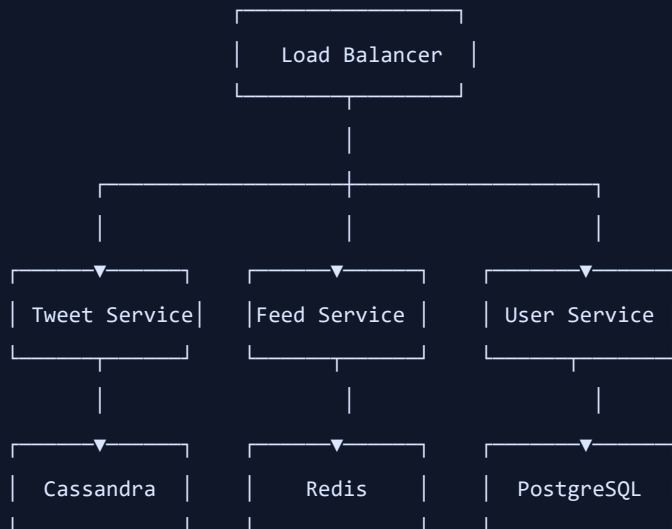
Non-Functional Requirements

- Low latency feed generation (<200ms)
- High availability
- Scalability (500M users, 300K QPS)
- Eventual consistency acceptable

Capacity Estimation

- **Users**: 500M daily active
- **Tweets**: 500M per day = ~6,000 QPS
- **Feed reads**: 300K QPS
- **Storage**: $500M * 280 \text{ chars} * 2 \text{ bytes} = \sim 280GB/day$

High-Level Design



Feed Generation Strategies

Fan-out on Write (Push Model)

- When user posts, push to all followers' feeds
- Fast reads, slow writes
- Good for users with few followers

Fan-out on Read (Pull Model)

- When user requests feed, pull from followees
- Fast writes, slow reads
- Good for celebrity accounts

Hybrid Approach (Recommended)

- Push for regular users (< 10K followers)
- Pull for celebrities (> 10K followers)
- Best of both worlds

Feed Generation Flow

1. User opens app, requests feed
2. Feed Service checks Redis cache
3. Cache miss: Query fan-out service
4. Fan-out service merges:
 - Pre-computed feeds from regular users
 - Real-time pulls from celebrities
5. Sort by timestamp/algorithm
6. Cache result in Redis
7. Return to client

Database Schema

```
-- Tweets Table (Cassandra)
CREATE TABLE tweets (
    tweet_id TIMEUUID,
    user_id BIGINT,
    content TEXT,
    media_urls LIST<TEXT>,
    created_at TIMESTAMP,
    like_count COUNTER,
    retweet_count COUNTER,
    PRIMARY KEY (user_id, created_at, tweet_id)
) WITH CLUSTERING ORDER BY (created_at DESC);

-- User Follows Table
CREATE TABLE user_follows (
    user_id BIGINT,
    followee_id BIGINT,
    created_at TIMESTAMP,
    PRIMARY KEY (user_id, followee_id)
);

-- Timeline Cache (Redis)
-- Key: timeline:{user_id}
-- Value: Sorted Set of tweet_ids
```

Timeline Cache (Redis)

```
# Add tweet to user's timeline
ZADD timeline:{user_id} {timestamp} {tweet_id}

# Get timeline
ZREVRANGE timeline:{user_id} 0 99

# Remove old tweets
ZREMRANGEBYRANK timeline:{user_id} 0 -1001
```

Scaling Considerations

1. **Database Sharding:** Shard by user_id
2. **Read Replicas:** For feed queries
3. **CDN:** For media content
4. **Rate Limiting:** Prevent abuse

Resources

- [How Twitter Uses Redis to Scale](#)
- [Twitter System Design](#)

YouTube / Video Streaming - System Design

YouTube / Video Streaming - System Design

Requirements

Functional Requirements

- Upload videos (intros, resolutions, various sizes)
- Stream videos (adaptive bitrate)
- Search, comments, likes
- Recommendations
- Channel subscriptions
- Live streaming (optional)

Non-Functional Requirements

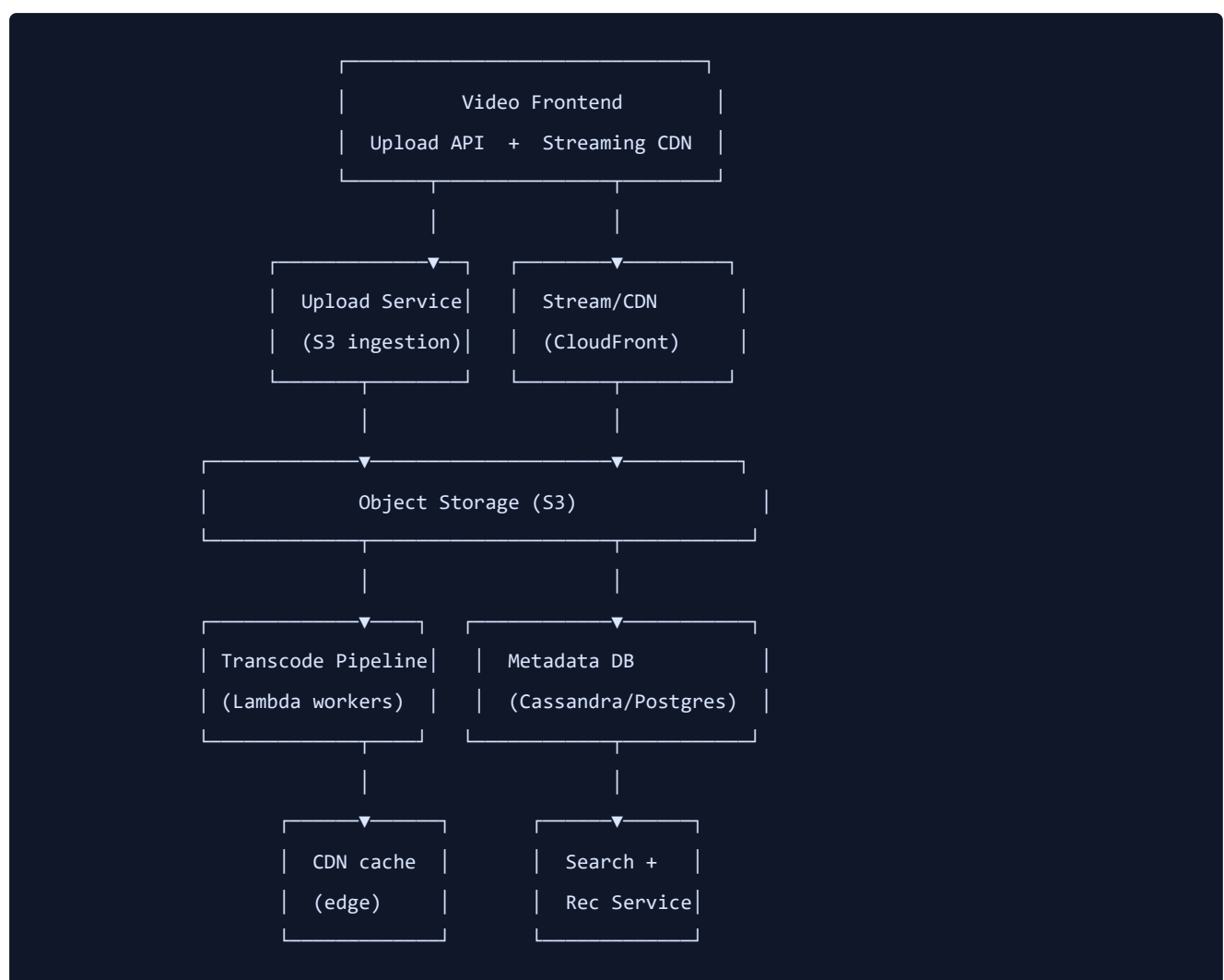
- Video delivery latency < 200ms (buffer start)
- Availability > 99.9%
- Scale: 1B+ users, 500hrs video uploaded every minute

- Bandwidth: massive - 1 video = many GB

Capacity Estimation

- **DAU:** 1B (2B+ total)
- **Uploads:** 500 hours per minute uploaded
- **Storage:** $500\text{hr}/\text{min} * 60\text{min} * 24\text{hr} = 720\text{K hours}/\text{day}$
- **Video size:** 1hr at 720p $\approx 500\text{MB}$
- **New storage/day:** $720\text{K} * 500\text{MB} = 360\text{TB}/\text{day}$
- **Data transfer:** $1\text{B users} * 20\text{ videos}/\text{day} * 25\text{MB}(5\text{min}) \approx 500\text{PB}/\text{day}$ worst case

High-Level Design



Upload Pipeline

1. Client requests upload URL (pre-signed S3)
2. Client uploads original .mp4 → S3 raw bucket
3. S3 event → trigger video processing (Lambda/K8s)
4. Processing stages:
 - a. Validation (metadata, virus scan, copyright check)
 - b. Transcode to multiple qualities:
360p / 480p / 720p / 1080p / 4K (HLS/DASH manifests)
 - c. Generate thumbnails (3-4 thumbnails, at t=1s, 10s, 30s)
 - d. Audio extraction (separate audio track)
 - e. On-demand: ML content tag / auto-caption
- Output:
HLS manifest (.m3u8) → lists .ts segments
DASH (.mpd) → .mp4 segments
All segments stored in formatted buckets
5. Update metadata DB (title, duration, dimensions, status)
6. Data available for streaming when 720p at least is ready

Transcoding Systems

System	Approach	Pros	Cons
Lambda	Serverless per job	scale on demand	cold starts, fragile long jobs
Batch (Elastic)	Server fleet	robust, K8s	cost, capacity planning
Spot instances	Cheap compute	low cost	preemption risk

Best practice: Queue-based processing (SQS) + worker pool.

Adaptive Bitrate Streaming (HLS)

```
Manifest (.m3u8)
├─ video_360p/segment_00001.ts
├─ video_360p/segment_00002.ts
├─ video_720p/segment_00001.ts
└─ video_1080p/segment_00001.ts
```

Client behavior:

1. Client fetches master playlist
2. Estimates available bandwidth
3. Selects appropriate quality

4. Buffers ~30-60 seconds
5. Switches quality based on buffer/bandwidth

Why segment-based? This is how Netflix/YouTube avoid down/up loading whole video. Users can jump to ranges; recovery from network blips is graceful.

Streaming (CDN)

Video segments stored across:

- Hot videos (top 1%): CDN edge cache
- Popular: region caches
- Long tail: origin S3 → CDN on demand

CDN caching policy:

- Cache-Control: public, max-age=60s (manifest)
- Segments: immutable (versioned), cache long
- Purge on delete/deny

Database Schema

```

-- Video Metadata (PostgreSQL/Cassandra)
CREATE TABLE videos (
    video_id UUID PRIMARY KEY,
    channel_id BIGINT,
    title VARCHAR(255),
    description TEXT,
    duration INT,          -- seconds
    s3_bucket TEXT,
    manifest_path TEXT,    -- main HLS playlist
    thumbnails JSONB,
    dimensions INT,        -- height px
    status VARCHAR(20),    -- uploading, processing, ready, failed
    view_count BIGINT DEFAULT 0,
    like_count BIGINT DEFAULT 0,
    created_at TIMESTAMPTZ,
    published_at TIMESTAMPTZ
);

-- Channel
CREATE TABLE channels (
    channel_id BIGINT PRIMARY KEY,
    owner_user_id BIGINT,
    name VARCHAR(255),
    subscriber_count BIGINT,
    created_at TIMESTAMPTZ
);

-- View events (write-heavy)
CREATE TABLE view_events (
    video_id UUID,
    user_id BIGINT,
    duration_watched INT,
    played_at TIMESTAMPTZ,
    -- partitioned by date
    PRIMARY KEY (video_id, played_at)
);

-- Comments (embedded or separate service)
CREATE TABLE comments (
    video_id UUID,
    comment_id UUID,
    user_id BIGINT,
    text TEXT,

```

```
created_at TIMESTAMPTZ,  
PRIMARY KEY (video_id, created_at, comment_id)  
);
```

View Counting

Increment → Redis counter (per video)
Periodic flush → DB
Real-time gauge: Redis counter gets
Cross-check/filter: dedupe by user, event log

Search & Recommendations

Search:

Video title/description/channel → Kafka → Elasticsearch index
Query → ES → Return video IDs → Join metadata → Serve

Recommendations:

View history → Kafka stream → ML service
Collaborative filtering / content similarity
Candidates → Candidate pool (Redis) → Rank → Top N

Live Streaming (Optional)

Streamer → Ingest (RTMP) → Transcoder (low latency) → CDN → Viewers (HLS/LL-HLS)

Ingest: dedicated server per streamer, bitrate capped
Transcoding: real-time per-segment
Latency targets: Live < 1s (WebRTC), Standard < 20s (HLS)

Note: This is why Twitch has a "channel ingest server" — per-streamer routes.

Failure Handling

Failure	Mitigation
Transcode fails	Retry 3x → Notify uploader, keep raw in S3

Failure	Mitigation
CDN down on one region	DNS failover to fallback CDN edge
Storage bucket corrupt	Versioned S3 uploads, checksums, replicate across AZ
DB spike on new upload	Geotargeted DB read replicas + queues

Interview Talking Points

1. **Pre-signed S3 upload:** no bottleneck at upload time
2. **Async transcoding:** queue-driven pipeline, notify when ready
3. **HLS/DASH segmentation:** adaptive quality over unpredictable networks
4. **CDN-first:** edge caching, near-zero bandwidth to origin for popular content
5. **Write-heavy counting:** Redis counters → flush to DB
6. **Long-tail cost:** origin on-demand + warm popular buckets

Resources

- [YouTube Engineering Blog](#)
- [Netflix's Video Encoding and Delivery](#)
- [Awesome HLS](#)
- [HTTP Live Streaming Spec \(RFC 8216\)](#)
- [Designing YouTube - Grokking](#)

09. Advanced Deep Dives

Google Docs/Real-time Collaboration - System Design

Google Docs/Real-time Collaboration - System Design

Requirements

Functional Requirements

- Create and edit documents
- Real-time collaboration (multiple users)
- Cursor presence and selection
- Version history
- Comments and suggestions
- Offline support

Non-Functional Requirements

- Low latency (<100ms for edits)
- High availability
- Conflict resolution
- Scalability (millions of concurrent users)

Capacity Estimation

- **Users:** 1B+ users
- **Concurrent editors per doc:** Up to 100
- **Edits per second:** 10K+ globally
- **Storage:** Documents + version history

Core Concepts

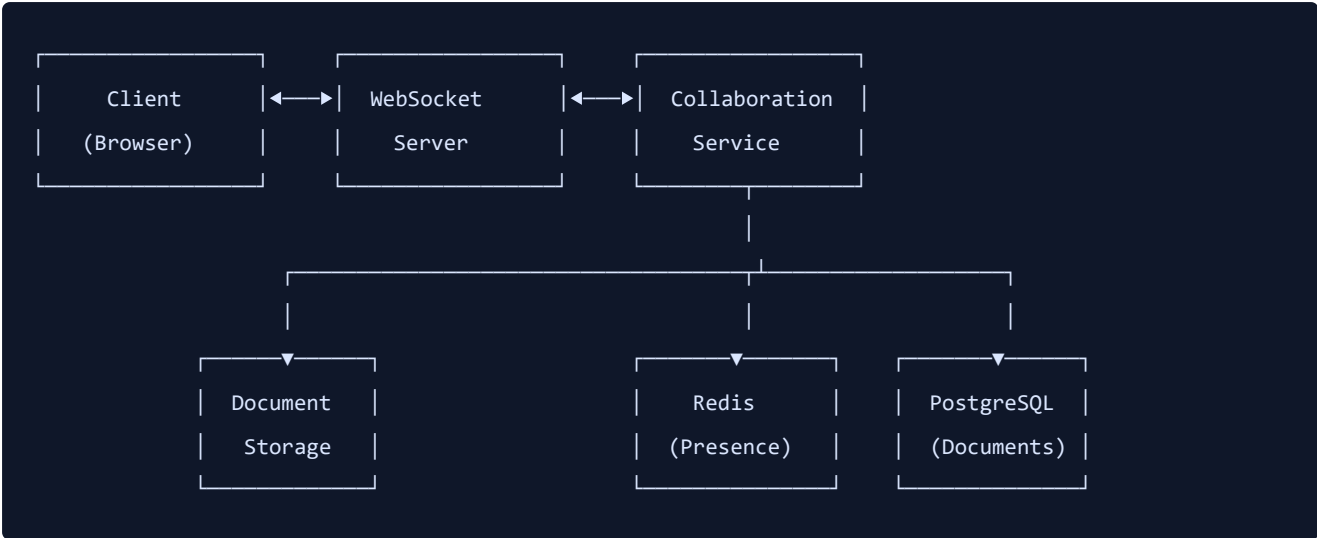
Operational Transformation (OT)

- Transform operations against concurrent operations
- Maintain consistency
- Server is source of truth

Conflict-free Replicated Data Types (CRDTs)

- Mathematical approach to consistency
- Operations can be applied in any order
- No central server needed
- Used in modern implementations

High-Level Design



OT vs CRDT Comparison

Aspect	OT	CRDT
Consistency	Strong	Eventual
Complexity	High	Medium
Server dependency	Required	Optional
Used by	Google Docs	Figma, Apple Notes

Collaboration Flow

1. User A types character 'X'
2. Client sends operation: `Insert(pos=5, char='X')`
3. Server receives operation
4. Server transforms against pending operations
5. Server broadcasts transformed operation to all clients
6. Clients apply operation locally
7. Server persists document state

Cursor Presence

Redis Hash:

```
user:{userId}:cursor = {  
  "docId": "abc123",  
  "position": 42,  
  "selection": {"start": 40, "end": 42},  
  "color": "#FF5733",  
  "name": "User A"  
}
```

Database Schema

```
-- Documents Table  
CREATE TABLE documents (  
  id UUID PRIMARY KEY,  
  title VARCHAR(255),  
  owner_id BIGINT,  
  content JSONB, -- Document content as JSON  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW(),  
  version BIGINT DEFAULT 0  
);  
  
-- Document Versions (for history)  
CREATE TABLE document_versions (  
  id BIGSERIAL PRIMARY KEY,  
  document_id UUID REFERENCES documents(id),  
  version BIGINT,  
  content JSONB,  
  created_by BIGINT,  
  created_at TIMESTAMP DEFAULT NOW()  
);  
  
-- Collaborators Table  
CREATE TABLE collaborators (  
  document_id UUID REFERENCES documents(id),  
  user_id BIGINT,  
  permission VARCHAR(20), -- owner, editor, viewer  
  added_at TIMESTAMP DEFAULT NOW(),  
  PRIMARY KEY (document_id, user_id)  
);
```

Offline Support

Local Operations

- Queue operations locally
- Apply immediately for responsiveness
- Sync when reconnected

Conflict Resolution

- Server transforms queued operations
- Client receives authoritative state
- Merge local changes

Performance Optimizations

1. **Debouncing**: Batch rapid keystrokes
2. **Compression**: Delta compression for operations
3. **Partitioning**: Shard documents across servers
4. **Caching**: Cache frequently accessed documents

Resources

- [Google Docs Architecture](#)
- [Operational Transformation Paper](#)
- [CRDTs for Collaborative Editing](#)
- [Figma's Multiplayer Technology](#)

Google Maps / Location Service - System Design

Google Maps / Location Service - System Design

Requirements

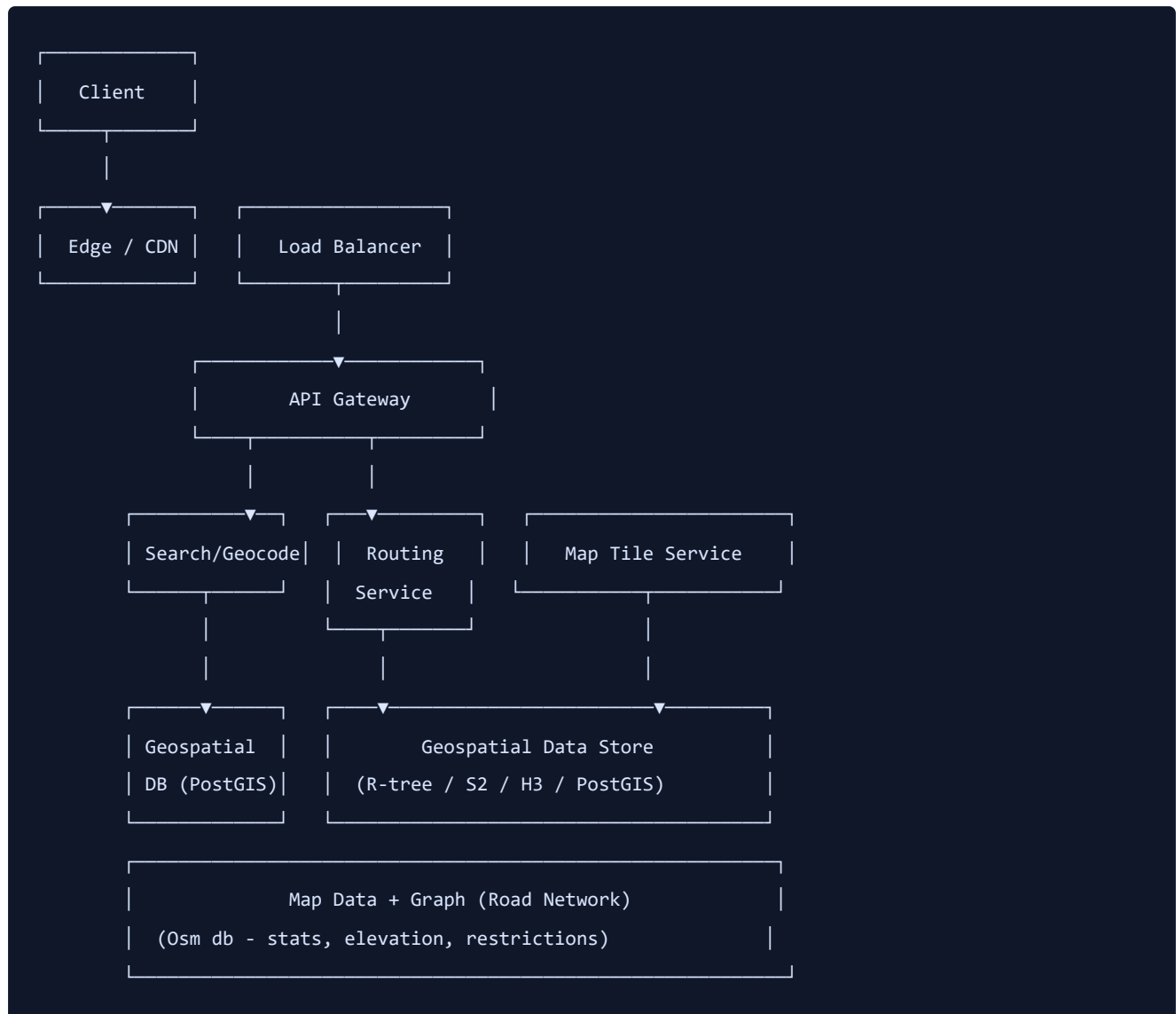
Functional Requirements

- Display map (tiles, zoom levels)
- Location search (places, addresses)
- Turn-by-turn navigation (A → B, with intermediate waypoints)
- Real-time traffic overlay
- ETA estimation
- Live location sharing (optional)

Non-Functional Requirements

- Map tile load < 100ms (CDN)
- Routing response < 1s
- Accuracy: 95%+ route correctness
- Availability: > 99.99% (maps are critical)
- Handle 1B+ users globally

High-Level Design



Key Components

1. Map Rasterization (Tiles)

Tiling System (XYZ)

```
Level 0: whole world = 1 tile (256x256px)
```

```
Level 1: 2x2 = 4 tiles
```

```
Level N:  $2^N \times 2^N$  tiles
```

```
Global zoom 0→22; typical map shows zoom 10-18; indoor zoom 22
```

```
URL: /tiles/z/x/y.png
```

Optimization:

- Pre-render tiles offline (batch, using GPU or software)
- CDN-cache: `Cache-Control: public, max-age=1year` (unchanged tile)
- Vector tiles (simplified geometry) → smaller, client renders
- Overlay layers: traffic, terrain rendered separately, composited client-side

2. Geocoding / Location Search

```
Query "Eiffel Tower, Paris" → Geocoder → lat/lng, bounding box, place ID
```

Storage:

- Place database (PostGIS / Elasticsearch)
- Syntax-based matching
- Fuzziness: edits (Levenshtein), prefix, n-grams
- Ranking: popularity, proximity, category

3. Routing / Turn-by-turn Navigation

Graph representation: Nodes = intersections, Edges = road segments.

Algorithms:

- Dijkstra (baseline, $O(V^2)$)
- A* (heuristic → fast for point-to-point)
- Bidirectional A* (faster large maps)
- Contraction Hierarchies (precomputed shortcuts → production Google uses this)
- ALT (Landmarks)

Production choices:

```
Precompute Contraction Hierarchies per region (offline, large compute).
```

```
Runtime: bidirectional search on compressed graph → near-instant.
```

```
Transit / walking / cycling vary graph weights (speed, elevation, mode restrictions).
```

Traffic Overlay & ETA

Live GPS traces → Kafka → Aggregator (5-min windows) →
Average speed per road segment → publish to tile server + routing

ETA = $\text{sum}(\text{road length} / \text{current avg speed} + \text{historical pattern})$
Historical pattern: day-of-week, hour-of-day (ML or lookup)

Data Model

```
-- Places (PostGIS)
CREATE TABLE places (
    place_id BIGINT PRIMARY KEY,
    name TEXT,
    lat DOUBLE PRECISION,
    lng DOUBLE PRECISION,
    category VARCHAR(50),
    popular_level INT DEFAULT 0,
    geom GEOGRAPHY(POINT, 4326) -- geospatial index
);
CREATE INDEX idx_places_geom ON places USING GIST(geom);

-- Road Network (graph)
CREATE TABLE roads (
    edge_id BIGINT PRIMARY KEY,
    start_node BIGINT,
    end_node BIGINT,
    length_m FLOAT,
    speed_limit_kmh FLOAT,
    road_class VARCHAR(20), -- motorway, primary, residential
    allowed_modes TEXT[]    -- driving, walking, cycling
);
CREATE INDEX idx_roads_nodes ON roads (start_node, end_node);

-- Geohash / S2 cells for proximity lookup
```

Proximity Queries

Problem: "Find 20 nearest coffee shops within 5km"

Indexes:

1. **R-tree** (PostGIS/PostgreSQL): spatial range query

2. **Geohash**: string prefix = geo region
3. **S2/H3**: hierarchical cells (Google uses S2, Uber uses H3)

```
SELECT * FROM places
WHERE ST_DWithin(geom, ST_MakePoint(-73.9,40.7)::geography, 5000)
ORDER BY ST_Distance(geom, ...) ASC LIMIT 20;
```

Live Location Sharing

```
Client → WebSocket/HTTP POST location updates (every 5-10s) →
  Location Service → Redis (pub/sub per trip) → Other participants

Consent/permission: access token per shared trip, TTL expiring.
```

Scaling & Failover

- Tiles: purely CDN cached; zero origin compute for hot tiles
- Search: Elasticsearch cluster scaled by region
- Routing: compute in regional clusters, cache routes (common origin/dest)
- Traffic: Kafka stream → regional aggregators
- Map data updates: nightly sync from OSM/Google; versioned tiles (cache invalidation via version string)

Interview Talking Points

1. **Tiling = content addressable cache** — the whole system rests on predictable CDN caching
2. **Precomputation** (Contraction Hierarchies) makes routing fast at runtime
3. **Geospatial indexes** (R-tree / S2 / Geohash) power proximity
4. **Data pipeline** (GPS traces → traffic → ETA) drives dynamic value
5. **Multi-modal routing** needs per-mode graph weighting

Resources

- [MIT: Geospatial Data Science](#)
- [PostGIS Documentation](#)
- [S2 Geometry Library \(Google\)](#)
- [H3 \(Uber\)](#)
- [OSRM - Open Source Routing Machine](#)

Twitter Clone - System Design

Twitter Clone - System Design

Requirements

Functional Requirements

- Post tweets (280 characters + media)
- Follow/unfollow users
- Home timeline (personalized feed)
- User profile
- Like, retweet, reply
- Trending topics
- Search

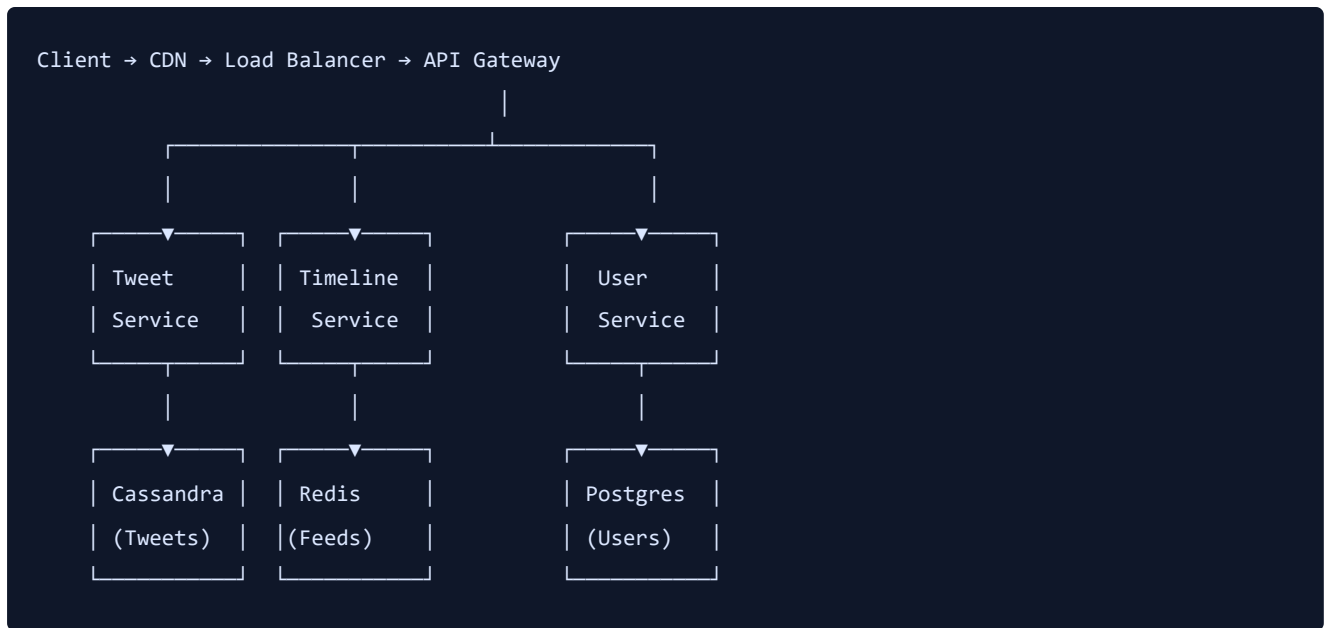
Non-Functional Requirements

- <200ms latency for timeline
- Eventual consistency for feed
- High availability
- Handle celebrities (millions of followers)
- Autonomous scaling

Capacity Estimation

- **Users:** 300M DAU (1.5B total)
- **Tweets/day:** 400M
- **Timeline reads:** 60K QPS peak
- **Followers:** Average 200 per user
- **Storage:** 400M * 280 bytes = 112GB/day text only

Architecture Overview



Tweet Service

POST /api/tweets

1. Validate tweet content
2. Store in Cassandra
3. Distribute to follower feeds
4. Update search index

GET /api/tweets/{id}

1. Check cache
2. Query Cassandra
3. Return tweet + metadata

Timeline Service

Fanout Strategy

- **Regular users** (<10K followers): Push to all followers
- **Celebrities** (>10K followers): Pull on-demand


```
def post_tweet(tweet_id, user_id, followers):
    if len(followers) < 10000:
        # Fanout-on-write
        for follower in followers:
            redis.zadd(f"timeline:{follower}",
                      {tweet_id: time.time()})
    else:
        # Fanout-on-read: store in celebrity feed
        redis.zadd(f"celebrity_feed:{user_id}",
                  {tweet_id: time.time()})
```

Timeline Generation

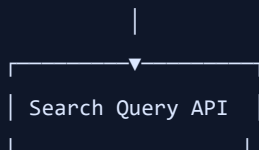
```
def get_timeline(user_id):
    timeline = redis.zrevrange(f"timeline:{user_id}", 0, 99)
    # If timeline empty, fallback to merge
    if not timeline:
        timeline = merge_following_feeds(user_id)
    return timeline
```

Trending Topics Service

1. **Ingestion:** Stream tweets via Kafka
2. **Processing:** Extract hashtags/words
3. **Aggregation:** Count frequency per time window
4. **Ranking:** Rank by score (count + recency)
5. **Caching:** Cache top trends every minute

Search Service

Tweets → Kafka → Index Pipeline → Elasticsearch



Database Schema

```

-- Tweets (Cassandra)
CREATE KEYSPACE twitter;
CREATE TABLE tweets (
    tweet_id TIMEUUID,
    user_id BIGINT,
    content TEXT,
    media_urls LIST<TEXT>,
    like_count COUNTER,
    retweet_count COUNTER,
    reply_to TIMEUUID,
    created_at TIMESTAMP,
    PRIMARY KEY (user_id, created_at, tweet_id)
) WITH CLUSTERING ORDER BY (created_at DESC);

-- Follows
CREATE TABLE follows (
    user_id BIGINT,
    followee_id BIGINT,
    created_at TIMESTAMP,
    PRIMARY KEY (user_id, followee_id)
);

-- Secondary index for:"user_id", index:"followee_id" to get followers

```

Media Storage

POST to Media Service → Get S3 pre-signed URL → Upload → Get permanent URL

Key Interview Points

1. **Fanout analysis:** Why hybrid approach?
2. **Cache optimization:** Redis sorted sets for timelines
3. **Celebrity problem:** How to handle @BarackObama's tweets
4. **Consistency model:** Eventual consistency for feeds
5. **Search indexing:** Real-time vs batch indexing

Resources

- [Twitter Architecture 2021](#)
- [Grokking System Design Interview](#)
- [System Design Interview - Alex Xu](#)

Web Crawler - System Design

Requirements

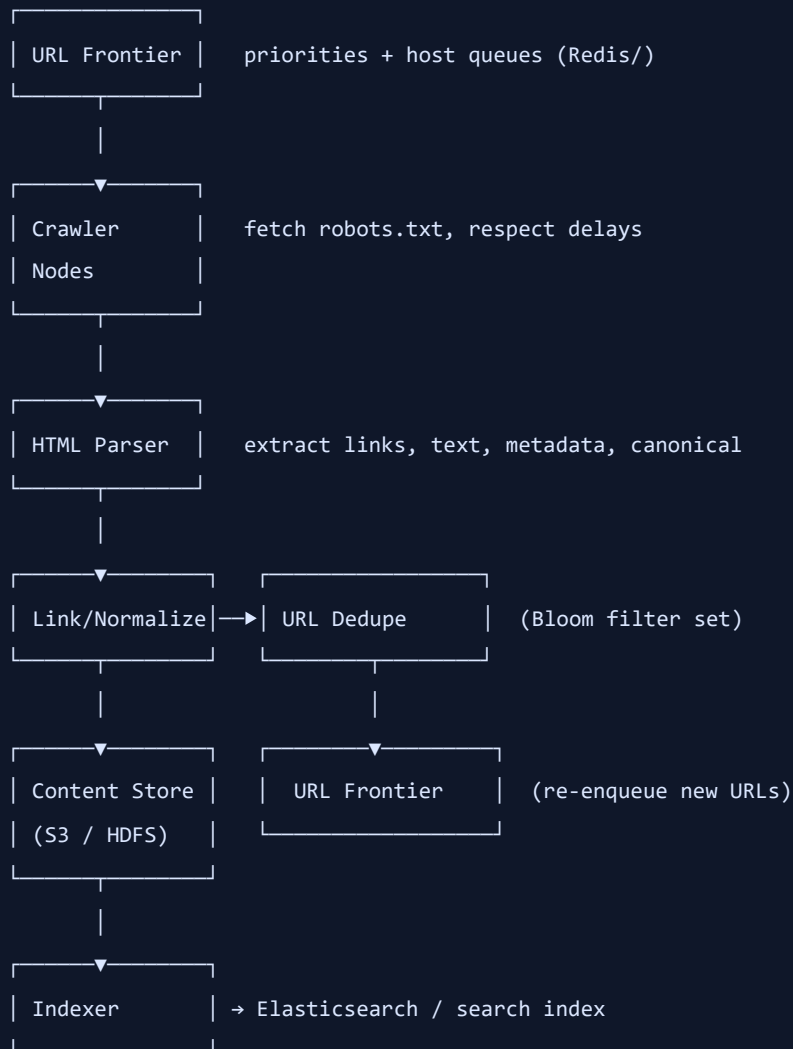
Functional Requirements

- Crawl the entire web (billions of pages)
- Extract content (HTML → structured data)
- Handle dynamic pages (JS-rendered)
- Deduplicate identical content
- Respect `robots.txt`
- Prioritize important URLs
- Refresh content periodically

Non-Functional Requirements

- Throughput: 50M pages/day minimum
- Politeness (1 request every 10-20s per host)
- Availability: 99.9%+ (crawl must be reliable)
- Scalability: add crawler nodes horizontally
- Robust to failures, retries, and network flakiness

High-Level Design



Components in Detail

1. URL Frontier (Design is key)

Must support:

- Prioritization: News > Snapshot ≈ Domain rank > Fresh (age)
- Host politeness: per-host queue (limit fetch rate)
- Avoid duplicate fetching
- Handle redirects and URL canonicalization

Implementation:

- Redis sorted sets: priority queues
- Per-host quota via token buckets
- Persistent frontier (survive restarts)

2. Fetching Node

Sequence per URL:

1. Check robots.txt (cached per host, honor crawl-delay)
2. Optional proxy pool (avoid IP bans)
3. HTTP GET with timeouts (e.g., connect 10s, read 30s)
4. Validate status (200ok, 301→follow redirect, 410→stop)
5. Validate content-type (HTML, feed, image, PDF)
6. Enforce size cap (e.g., max 2MB)
7. Store raw + metadata (headers, timestamps, fetch result)

3. Parsing & Normalization

Parse:

- Extract <link>, <a>, srcset, canonical, title, meta, og tags
- Extract structured data (JSON-LD, microdata)
- Text extraction (tags removed, whitespace normalized)
- Detect language

Normalize URL:

- Remove fragments (#), trailing slash, default ports
- www vs non-www (choose canonical)
- Case-normalize path, resolve relative to absolute
- Strip tracking params (?utm_, &fbclid) unless required

4. Deduplication

Why: The web has massive duplication (shingles, boilerplates, CDNs, mirrors).

Approaches:

- SimHash / MinHash for near-duplicate content
- Content hash (SHA-256 of normalized text)
- URL set (Bloom filter + Redis) to avoid refetching

Politeness + quality: keep one canonical, dedupe others.

5. Content Store

- Raw HTML + URL → object storage (S3/parquet, partitioned by crawled_date)
- Processed structs → columnar store
- Live index → Elasticsearch (title, text, URL, dates, language)
- Snapshot history for freshness/re-crawl comparisons

6. Politeness & Ethical Crawling

- Robots.txt: parse and honor per host
- Crawl-delay: min 1 request per N seconds per host (10-20s typical)
- Never hammer same host
- Throttle per country / IP segment to avoid bans
- Respect rate limits, optional listen to sitemaps (fast + sanctioned)
- Include Crawler identity in User-Agent

Priority Handling (Popular vs Fresh)

```
Priority score =  
0.8 * domain_rank (PageRank-like, recency-weighted)  
+ 0.15 * link_popularity (incoming links)  
+ 0.05 * freshness_boost (recently changed / breaking news)
```

Needs a priority queue scheduler (e.g., Kafka or Redis ZSET) with re-crawl cycles for high-value pages.

Refresh Policy

- News sites: re-crawl every few minutes
- Blogs: hourly
- Static docs: daily
- Long-tail pages: weekly/monthly

Track last_crawled_at, content_hash, change detection.

Scaling the Crawler

Component	Scale method
Frontier	Shard by host-hash → distribute queues
Fetch nodes	Horizontal (stateless workers, pull from queue)
Parser	In-place on workers or separate pool
Dedup	Bloom filter in fast cache; periodic sync
Store	Object storage (S3) + partition by date

Crawl rate = workers * req/min/worker

For 50M pages/day + 2-min cycle → ~35K requests/sec → ~1,400 workers.

Failure Handling

Failure	Mitigation
Timeout / connection reset	Retry with exponential backoff (max 3)
HTTP 500/503	Retry later (respect Retry-After header)
Encoding errors	Detect via charset, fallback to control stream
Throttled (429)	Back off per host, distribute across proxies
Crawler crash mid-batch	Frontier is durable; re-crawl from checkpoint

Interview Talking Points

1. **URL frontier is the heart:** prioritize + politeness + dedup
2. **Robots.txt first:** ethical crawl, low ban risk
3. **Bloom filter dedup:** memory-efficient, false positives acceptable
4. **Edge caching:** robots.txt cache per host
5. **Pipeline isolation:** fetch/parse/store/components scale independently
6. **Recency tracking:** change-rate adaptive re-fetch

Resources

- [The Anatomy of a Large-Scale Hypertextual Web Search Engine \(Google/Brin\)](#)
- [W3C Robots.txt Spec](#)
- [Scrapy \(Python crawler framework\)](#)
- [Apache Nutch \(Hadoop-based crawler\)](#)
- [Crawler design - MIT 6.824 / TJBot](#)

10. Interview Prep Playbook

System Design Interview Playbook

System Design Interview Playbook

The 4-Step Framework

Every system design interview should follow this structure:

Step 1: Requirements (5 min)

-  Ask clarifying questions
-  Identify functional requirements
-  Identify non-functional requirements
-  Agree on scope
-  DON'T jump to solution

Step 2: Estimation (5 min)

-  Calculate QPS, storage, bandwidth
-  State assumptions clearly
-  Use round numbers ($2^{10} = 1K$, $2^{20} = 1M...$)
-  Don't over-optimize accuracy

Step 3: High-Level Design (15 min)

-  Draw main components
-  Explain data flow
-  Identify bottlenecks
-  Discuss trade-offs
-  Keep it simple initially

Step 4: Deep Dive (15 min)

-  Pick 1-2 areas the interviewer highlights
-  Detail the chosen area
-  Discuss scaling
-  Mention failure modes
-  Don't detail everything

Communication Framework for Each Component

For every component you introduce, cover:

1. **What:** What it does
2. **Why:** Why you chose it
3. **Trade-offs:** What you gave up
4. **Alternatives:** What else you considered

Interview Checklist

Before the Design

- ☐ Clarify user count (1M vs 100M changes everything)
- ☐ Clarify read/write ratio
- ☐ Clarify consistency requirements
- ☐ Ask about data size
- ☐ Ask if latency matters

During the Design

- ☐ Draw a clear diagram
- ☐ Label every component
- ☐ Show data flow arrows
- ☐ Mention the DB schema
- ☐ Identify the bottleneck
- ☐ Discuss the cache strategy

What Interviewers Look For

1. **Structure:** Organized thinking, no random jumping
2. **Communication:** Clear explanations of concepts
3. **Trade-offs:** Ability to compare options
4. **Depth when needed:** Know when to go deep
5. **Breadth:** Know multiple solutions/patterns

Top Interview Mistakes

1. Not Clarifying Requirements

✗ "Let me design Twitter" → jumps to feed ✓ "Who are the users? How many? What scale? Mobile or web?"

2. Not Estimating

✗ No numbers at all ✓ "100M DAU, 10% post daily = 10M posts/day → 115 QPS"

3. Overshadowing Details

❌ Perfecting edge cases instead of finishing the design ✅ Complete the design first, improve later

4. Ignoring Trade-offs

❌ "Use Kafka everywhere" ✅ "Kafka for async processing, but NO for request-response via SQS"

5. Not Mentioning Bottlenecks

❌ Design works for 1 user ✅ "The DB write is our bottleneck; here's how we shard"

6. Forgetting Failure Modes

❌ Single point of failure everywhere ✅ "If Redis fails, we fall back to DB with degraded response"

Approaching Design Questions

Pattern: "Design X for Y Users"

Common systems interviewers ask:

- Twitter, Facebook, Instagram → **Read-heavy feed systems**
- WhatsApp, Slack → **Real-time messaging**
- YouTube, Netflix → **Video streaming**
- Google Docs → **Collaboration + OT/CRDT**
- Uber, Lyft → **Location + matching**
- Amazon, eBay → **E-commerce + payments**
- Google Search → **Search + web crawler**

Prioritization for Each Problem

1. **Data flow**: What is core? What's optional?
2. **Read-heavy**: Cache, CDN, read replicas
3. **Write-heavy**: Queues, async processing, sharding
4. **Real-time**: WebSockets, push infrastructure
5. **Consistency**: When can we be eventual?

Estimation Cheat Sheet

1M requests/day = ~12 QPS

100M requests/day = ~1,200 QPS

200M requests/day = ~2,400 QPS

Storage for 1 item:

Tweet text: 280 bytes

User profile: 1KB

Photo: 500KB

Video (1hr): 1GB

Bandwidth:

1M photos/day * 500KB = 500GB/day ≈ 5.7MB/s

Shard sizes:

PostgreSQL: ~1TB practical limit

Cassandra: ~1PB+ with cluster

Redis: ~64GB per node (RAM limit)

Quick Reference: Component Selection

Database Selection

Use Case	Choose	Why
Transactions	PostgreSQL	ACID, mature
User profiles	PostgreSQL/MongoDB	Flexible schemas
Feeds	Cassandra	Write-heavy, time-series
Cache	Redis	Speed, data structures
Search	Elasticsearch	Full-text, filters
Analytics	ClickHouse/BigQuery	OLAP, aggregations

When to Use Queues

- ☒ Async jobs (email, notification)
- ☒ Decoupling services
- ☒ Load leveling
- ☒ Event streaming
- ☒ Request-response (latency)
- ☒ Sequential processing (ordering)

The "Signal Words" That Change Designs

When interviewer says	It means
"Like Twitter"	read-heavy, feed generation
"at scale"	horizontal scaling, sharding
"real-time"	WebSocket, push
"billions"	aggressive caching, tiered storage
"eventually"	eventual consistency is OK
"exactly once"	idempotency, transactions
"globally"	multi-region, edge, CDN
"hot" content	cache, CDN, priority queues

Where to Practice (ranked by value for money)

Platform	What you get	Cost	Best use
Pramp	Scheduled peer mock interviews	Free	#1 free option — book 2-3/week, alternate roles
interviewing.io	Anonymous live mocks; recorded real company sessions	Freemium	Pressure reps + learn by watching recorded passes
LeetCode (System Design tag)	Text designs + community answers	Paid	See what "model" answers look like; read AFTER self-attempt
Grokking SD Interview	16 guided problems	Paid	Reference solutions after your attempt
ByteByteGo	Diagrams, course, mock + assessment	Paid	Visual memory aids + structured mocks
YouTube (Gaurav Sen / ByteByteGo / codeKarle)	Full walkthroughs	Free	Watch AFTER attempting — never before

The rule that decides outcomes: practice *producing* designs, not *recognizing* them. Every session is: pick problem → 30 min out loud → compare → fix one gap.

Frequently Asked Questions — Bank (work top→bottom)

Full difficulty tiers, extensions, and alternate takes are in `../PRACTICE.md`. Master the MUST set (#1-5) cold — they cover 70%+ of interview questions and every core pattern.

#	Problem	Priority	Difficulty	Study here
1	URL Shortener	🔥 MUST	Easy	<code>../09-Projects/URL-Shortener/</code>
2	Rate Limiter	🔥 MUST	Easy	<code>../09-Projects/Rate-Limiter/</code>

#	Problem	Priority	Difficulty	Study here
3	News Feed (Twitter/FB)	🔥 MUST	Medium	../10-LargeScale/Twitter-Feed/
4	Chat / WhatsApp / Slack	🔥 MUST	Medium	../09-Projects/Chat-System/
5	Instagram / Photo Sharing	🔥 MUST	Medium	../10-LargeScale/Instagram/
6	Notification System	High	Medium	../09-Projects/Notification-System/
7	YouTube / Video Streaming	High	Hard	../10-LargeScale/YouTube/
8	Google Docs / Collab	High	Hard	../11-Advanced/Google-Docs/
9	Google Maps / geospatial	Med	Hard	../11-Advanced/Google-Maps/
10	Web Crawler	Med	Hard	../11-Advanced/Web-Crawler/
11	E-commerce / Amazon	High	Hard	See PRACTICE.md #12
12	Uber / Lyft matching	Med	Hard	See PRACTICE.md #10
13	Google Search	Med	Hard	See PRACTICE.md #15
14	Typeahead / Autocomplete	Med	Medium	See PRACTICE.md #14
15	Distributed Lock	Low	Hard	See PRACTICE.md #16
16	Leaderboard	Low	Medium	See PRACTICE.md

Practice Routine (proven weekly loop)

Daily (45-60 min):

1. 15 min: Requirements + estimates drift on a 3×3 note
2. 25 min: Design one problem out loud (voice-memo it)
3. 5 min: Self-scoring on the 5 interview axes

Weekly (2-3h):

- 2 mocks (Pramp / interviewing.io)
- 1 problem re-done cold (no notes)
- 1 "wildcard" unseen problem
- Debrief each: write down exactly 1 fix per mock

The 5 interview axes (score yourself 1-5 each session)

1. **Requirements** — did I clarify scope? (users, reads/writes, consistency)
2. **Estimation** — did I produce numbers? (QPS, storage, latency)
3. **Architecture** — clear diagram, labeled, simple first
4. **Trade-offs** — did I say what I gave up and alternatives?
5. **Failure handling** — did I design for outage?

Interview-Day Checklist (rehearse the night before)

- ☐ 4-step flow timing internalized (5/5/15/15/5 min framework)
- ☐ Numbers sheet in head (recite cold)
- ☐ Cheat-sheet one-pager at hand
- ☐ 2 warm-up designs already rehearsed
- ☐ Voice-crisp: talk continuous, use "X gives A, costs B, alternative C"

Resources

- [Master Study Plan \(start here\)](#)
- [System Design Interview - Alex Xu](#)
- [Pramp Free Mock Interviews](#)
- [Interviewing.io](#)
- [ByteByteGo Interview Prep](#)
- [Grokking the System Design Interview](#)